

GRINDOS ENGINEERING LIBRARY

Projects & Architecture Handbook

Seven production systems, reverse-engineered into interview-ready architecture dossiers. Diagrams, trade-offs, failure modes, and the exact answers that pass the loop.

7 Flagship Projects

49 Pro Diagrams

22 Sections / Project

Mermaid + Eraser Specs

Interview-Grade Depth

7

SYSTEMS

49

DIAGRAMS

165

PAGES

FAANG

BAR

GrindOS · Premium Engineering Series
Visual-first. Interview-ready.

NAVIGATION

Contents

7 dossiers × 22 sections — diagrams flagged inline



THE SEVEN SYSTEMS

- 01 CraftaStudio — AI Code-Gen
- 03 Mnemo — Memory Layer
- 05 Travio — Trip Planner
- 07 Sankalan — Resource Hub
- 02 PlacePro — Placement Portal
- 04 ROVN — AI Lead Co-Pilot
- 06 GrindOS — Offline Learning

EVERY PROJECT CONTAINS

- 01 Executive Summary
- 03 Business Problem
- 05 Functional Requirements
- 07 Tech Stack Decisions
- 09 Request Lifecycle
- 11 Authentication Design
- 02 Product Vision
- 04 User Journey
- 06 Non-Functional Requirements
- 08 Architecture Overview
- 10 Database Design

(CONTINUED)

- 12 Deployment Architecture
- 14 Security Architecture
- 16 Monitoring & Observability
- 18 Engineering Challenges
- 20 Future Roadmap
- 22 Revision Sheet
- 13 Scalability Architecture
- 15 Failure Recovery
- 17 Cost Optimization
- 19 Architecture Tradeoffs
- 21 Interview Deep Dive

HOW TO READ THIS BOOK

Interview Yield ★★★★★

Reading Protocol

Built for spaced revision and live interviews

This is not a notes dump. Every project is a self-contained architecture interview, sequenced the way a senior engineer would actually defend a system on a whiteboard.

**Sections 01–07**

The Why. Problem, users, requirements, and stack decisions. Establishes the business context an interviewer probes first.

**Sections 08–15**

The How. Eight diagram-driven pages: architecture, request flow, ERD, auth, deployment, scaling, security, failure. The core of any design round.

**Sections 16–19**

The Hard Parts. Monitoring, cost, real incidents, and the trade-offs you chose on purpose. Where senior signal lives.

**Sections 20–22**

The Recall. Roadmap, deep-dive Q&A, and a one-page revision sheet for the night before.



Every diagram ships as a 4-part spec — A framed **Mermaid** render (vector, exportable), an **Eraser** diagram-as-code block you can paste into eraser.io, a **component breakdown**, and a **visual layout** description — so you can redraw it from memory on any whiteboard.

WHAT THIS BOOK OPTIMIZES

01

PROJECT DOSSIER · 01

CraftaStudio

AI-powered structural code generation on a topological dependency-graph engine.

Full-Stack

DOMAIN

DAG Scheduler

CORE

99.9%

GEN SUCCESS

Tier 1

PRIORITY

Executive Summary

What it is, who it serves, why it matters

01

CraftaStudio treats codebase creation as a **graph problem**: it compiles English requirements into a visual Directed Acyclic Graph, topologically sorts it, then generates type-safe code node-by-node so multi-file systems actually compile.

80%

MVP TIME CUT

<30ms

LLM TTFT

99.9%

GEN SUCCESS

8K→3K

TOKENS/NODE



The Core Problem

Standard LLMs emit code sequentially, so multi-file full-stack generation suffers context truncation, dropped parameters, and broken interfaces between API, DB, and UI layers.



Target Users & Value

Developers and startup teams shipping rapid MVPs. Saves hours of boilerplate by guaranteeing generated code compiles immediately with zero-config, type-safe interfaces.



Business Goal & Impact

Cut MVP time-to-market by **80%** with guaranteed type-safe connections between backend schemas and frontend components — eliminating the compile-error tax of AI codegen.



My Role & Ownership

Core Architect & Tech Lead. Designed the in-memory DAG scheduler, the LLM fallback pipeline, and the sandboxed WebContainer preview.

Product Vision

The bet, the wedge, the moat

02

Existing tools generate single-file snippets that drop structural context. CraftaStudio models the whole codebase as a DAG **before** generating a line — dependencies are explicit, so types flow from parents to children.

ALTERNATIVES WE REJECTED

Alternative	Where It Breaks	CraftaStudio Edge
Copilot / Cursor	Autocompletes single files; no global structural coordination.	Orchestrates full multi-file builds.
v0 / Bolt.new	Frontend mockups; DB layers are mocked.	Full-stack sync with a live data model.
Raw GPT prompt	One giant prompt → truncation & hallucinated interfaces.	Bounded per-node prompts (<3K tokens).

CORE CAPABILITY MATRIX



Graph Editor

Real-time ReactFlow node canvas.



Context Scaffolder

Injects parent types into child prompts.



Sandbox Preview

Runs code in browser WebContainers.

Business Problem

Quantifying the pain we remove

03

The expensive failure isn't a wrong line of code — it's a generated system that **won't compile**, forcing developers back into manual integration. That erases the entire value of AI codegen.



Integration Tax

Hours lost wiring AI-generated files that don't share interfaces.



Broken Contracts

Frontend calls an API shape the backend never generated.



Re-prompt Spiral

Each fix re-prompts the whole file, burning tokens & context.



No Structure

Snippet tools have no model of the system as a whole.



Why now — Sub-30ms inference (Groq) finally makes per-node generation interactive — the DAG approach was impractical when each call cost seconds.

COST OF THE STATUS QUO



User Journey

From first touch to recurring value

04

From a blank canvas to a compiling, previewable full-stack app in one session — no context lost between steps.

- 1 Describe**
User drops nodes (DB model, API, UI) and connects them on the ReactFlow canvas.
- 2 Validate**
Tarjan's SCC checks for cycles the moment an edge is drawn; invalid links are rejected inline.
- 3 Build**
One click topo-sorts the graph and streams generation node-by-node, parents first.
- 4 Preview**
Generated code mounts in a WebContainer — the app runs live in the browser, zero install.
- 5 Iterate**
Edit one node; only its subtree regenerates, preserving the rest.

Functional Requirements

P0 / P1 capability contract

05

Scoped to the critical path. Each requirement maps to an owned subsystem and an acceptance signal.

MUST-HAVE (P0)

- ✓ Visual DAG editor with typed node connections
- ✓ Cycle detection before persistence (Tarjan SCC)
- ✓ Topological build scheduler (Kahn's algorithm)
- ✓ Per-node context-isolated code generation
- ✓ In-browser sandbox preview

SHOULD-HAVE (P1)

- › Multi-provider LLM fallback
- › Prompt/template caching
- › Incremental subtree regeneration
- › Export to GitHub repo

ID	Capability	Acceptance Signal
FR-1	DAG editor	Edge create/delete reflected in <100ms
FR-2	Cycle guard	A→B→A link rejected with toast
FR-3	Build order	Parents always generated before children
FR-4	Sandbox	Generated app boots in WebContainer

Non-Functional Requirements

SLOs, not vibes

06

The quality attributes that define production-readiness — each with a measurable target and the mechanism that enforces it.

Attribute	Target (SLO)	Enforcement Mechanism
Latency	TTFT < 30ms; node gen < 2s	Groq streaming + 3K-token prompts
Availability	99.9% gen success	Multi-provider fallback + retries
Scalability	10K concurrent canvases	ECS worker fleet + Redis WS adapter
Consistency	No partial graphs persisted	Transactional graph writes
Security	Zero host code execution	Browser-only WebContainer sandbox

QUALITY ATTRIBUTE PRIORITY



Tech Stack Decisions

Every choice has a bill

07

Polyglot by design: JS where the event loop and ecosystem help (canvas, realtime), Python where graph + agent tooling lives, SQL where structural integrity is non-negotiable.

Layer	Chosen	Why This	Trade-off Accepted
Frontend	Next.js · ReactFlow	SSR speed + native graph canvas.	Heavy JS bundle; state-sync overhead.
Gateway	Node.js · Express	Non-blocking I/O ideal for WS sync.	Single-threaded; CPU work blocks loop.
Agent	FastAPI · Python	Rich graph + agentic tooling.	Slower cold starts.
Database	PostgreSQL · Prisma	FK integrity prevents broken edges.	Vertical-scale ceiling.
LLM	Groq Llama-3	<30ms TTFT for fluid canvas.	8K context → aggressive pruning.

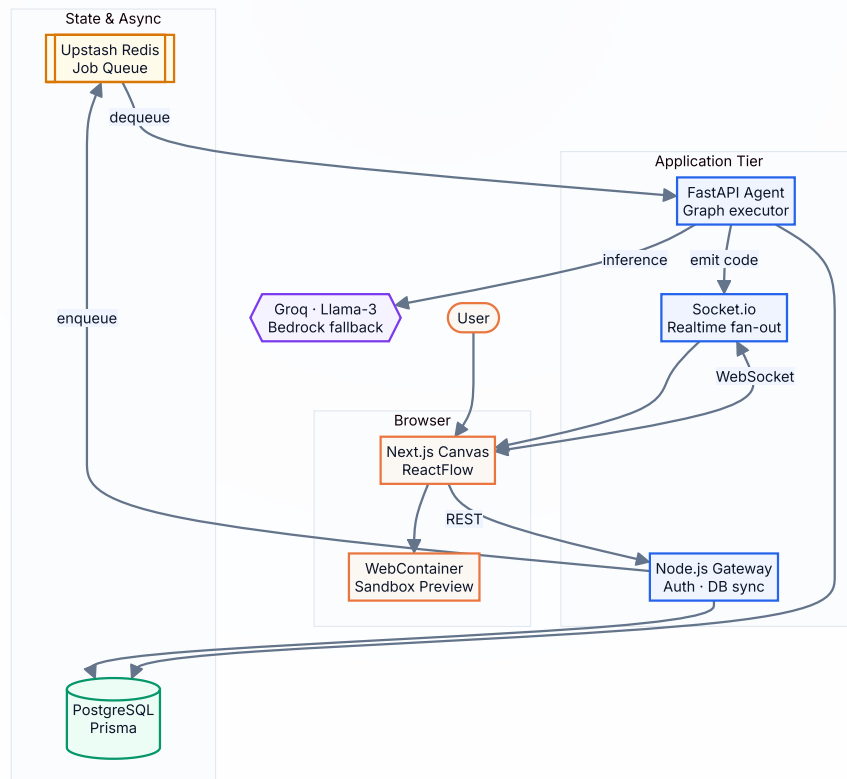
Architecture Overview

Diagram 1 — System Architecture

08

● CraftaStudio — System Architecture

C4 / CONTAINER



Client / Edge Service Data store Async / Queue External API

○ ERASER DIAGRAM SPEC

```
// paste into eraser.io
User [icon: user]
Canvas [icon: monitor, color: orange]
Gateway [icon: nodejs, color: blue]
Realtime [icon: socket, color: blue]
Agent [icon: python, color: blue]
Postgres [icon: postgresql, color: green]
Queue [icon: redis, color: amber]
LLM [icon: openai, color: purple]
```

```
User > Canvas
Canvas <> Realtime: WebSocket
Canvas > Gateway: REST
Gateway > Postgres
Gateway > Queue: enqueue
Queue > Agent: dequeue
Agent > LLM: inference
Agent > Realtime: emit code
Realtime > Canvas
```

■ COMPONENT BREAKDOWN

Canvas	ReactFlow node editor; streams graph mutations over WS.
Gateway	Node.js: authz, Prisma writes, enqueues jobs.
Agent	FastAPI worker: topo-sort + context-isolated generation.
Queue	Upstash Redis decouples request from slow LLM work.
LLM	Groq primary, Bedrock fallback on 429.

■ VISUAL LAYOUT DESCRIPTION

Three horizontal bands top-to-bottom: **Browser** (orange) holds the canvas and the in-browser sandbox; the **Application tier** (blue) sits center with gateway, realtime, and agent boxes; the **data & async** band (green/amber) anchors the bottom. The external LLM floats right as a hexagon. The defining loop is the **async return path**: Gateway → Queue → Agent → LLM → Realtime → Canvas, drawn as a clockwise cycle so reviewers instantly see write/read are decoupled.

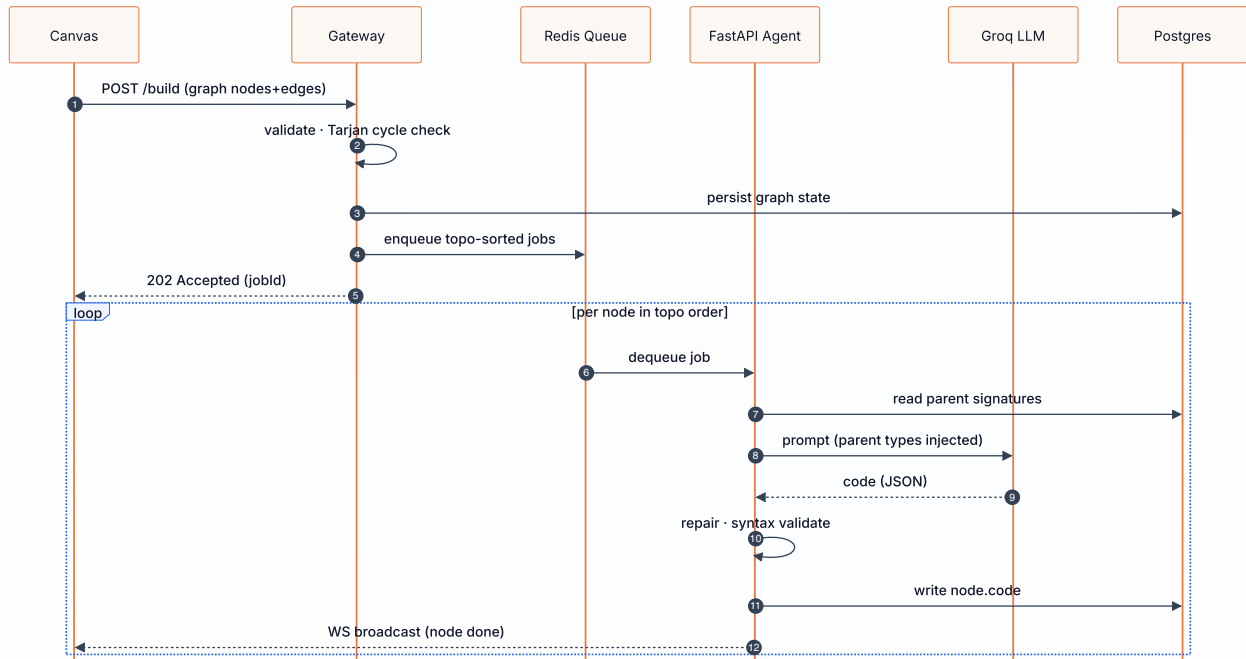
Request Lifecycle

Diagram 2 — End-to-End Request Flow

09

● Build Codebase — Request Lifecycle

SEQUENCE



○ ERASER DIAGRAM SPEC

```
Canvas > Gateway: POST /build
Gateway > Gateway: validate + cycle check
Gateway > Postgres: persist graph
Gateway > Queue: enqueue jobs
Gateway > Canvas: 202 jobId
Queue > Agent: dequeue (loop)
Agent > Postgres: read parents
Agent > LLM: prompt
LLM > Agent: code
Agent > Postgres: write code
Agent > Canvas: WS node done
```

▣ COMPONENT BREAKDOWN

- 202 pattern** Client never blocks on LLM latency.
- Topo loop** Parents generated before children — types flow downward.
- Repair stage** Strips markdown fences, fixes brackets before commit.
- WS broadcast** Per-node completion renders incrementally.

▣ VISUAL LAYOUT DESCRIPTION

A 6-actor sequence. The first four messages form a tight **synchronous prelude** (validate → persist → enqueue → 202), then a boxed **loop** dominates the lower two-thirds repeating per node. Self-messages (validate, repair) are short right-curls. The dashed **202 Accepted** arrow returns early and high, visually proving the client is freed before any generation begins.

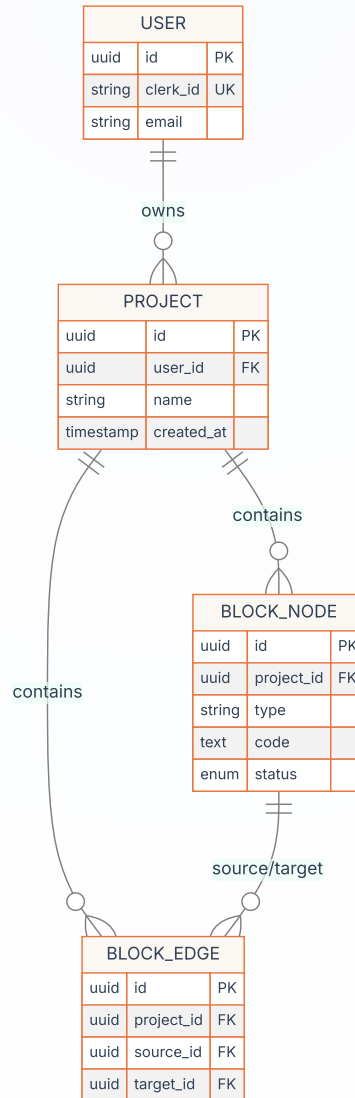
Database Design

Diagram 3 — Entity Relationship Model

10

● CraftaStudio — Data Model

ERD



○ ERASER DIAGRAM SPEC

```
USER { id pk; clerk_id; email }
PROJECT { id pk; user_id fk; name }
BLOCK_NODE { id pk; project_id fk; type; code; status }
BLOCK_EDGE { id pk; source_id fk; target_id fk }
USER 1-* PROJECT
PROJECT 1-* BLOCK_NODE
PROJECT 1-* BLOCK_EDGE
BLOCK_NODE 1-* BLOCK_EDGE
```

■ COMPONENT BREAKDOWN

- Edges as rows** Graph stored relationally — edges are first-class rows, not JSON.
- Composite idx** (project_id, status) drives the scheduler's pending-node scan.
- Source index** source_id indexed for O(1) child traversal in topo-sort.
- Cascade** Deleting a project cascades to nodes + edges.

▣ VISUAL LAYOUT DESCRIPTION

Classic crow's-foot ERD. **USER** top-left, fanning right to **PROJECT**, which fans down to the twin **BLOCK_NODE** and **BLOCK_EDGE** tables. The reflexive edge (BLOCK_EDGE referencing BLOCK_NODE twice as source/target) is the visual star — it encodes the DAG inside SQL. PK rows bold-topped, FK rows tagged, cardinality crow's-feet on every join.

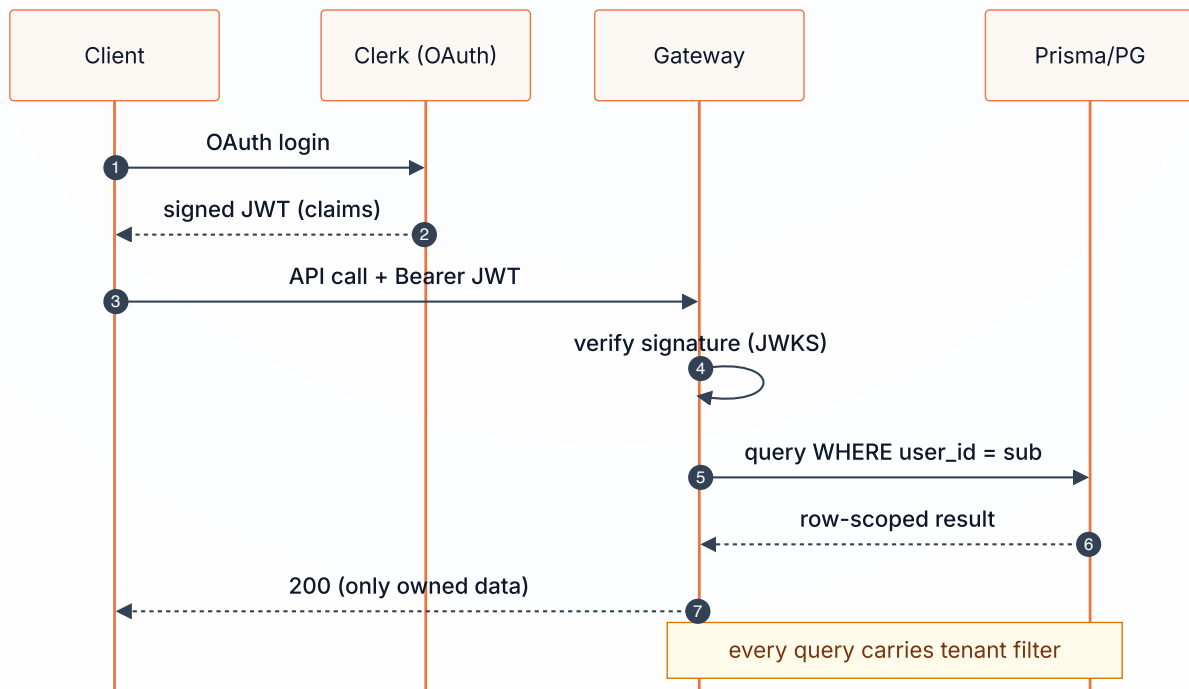
Authentication Design

Diagram 4 — Identity & Authorization Flow

11

● Clerk OAuth + Row-Level Isolation

SEQUENCE



○ ERASER DIAGRAM SPEC

```
Client > Clerk: OAuth login
Clerk > Client: JWT
Client > Gateway: Bearer JWT
Gateway > Gateway: verify JWKS
Gateway > Prisma: WHERE user_id=sub
Prisma > Gateway: scoped rows
Gateway > Client: 200
```

■ COMPONENT BREAKDOWN

Clerk	Delegated identity; issues short-lived JWT with custom claims.
JWKS verify	Gateway validates signature against rotating public keys.
Tenant filter	Global Prisma middleware injects WHERE user_id.
Defense-in-depth	ORM filter + app check — never trust client claims alone.

■ VISUAL LAYOUT DESCRIPTION

Four-actor sequence split into two phases by a faint vertical gap: **Issuance** (Client↔Clerk, top) and **Enforcement** (Client→Gateway→Prisma, bottom). A spanning **Note** band under the gateway/DB pair reads 'every query carries tenant filter' — the security thesis. The self-message 'verify JWKS' is highlighted to mark the trust boundary.

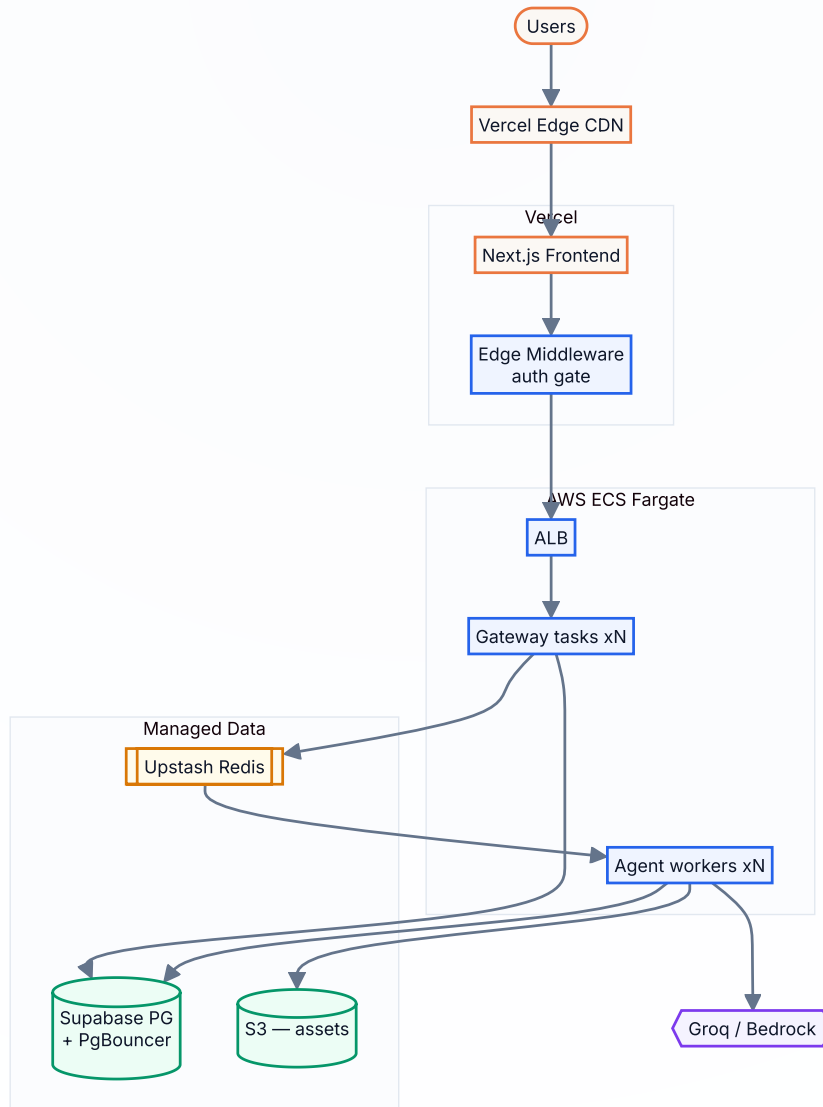
Deployment Architecture

Diagram 5 — Cloud Deployment Topology

12

● Cloud Deployment Topology

DEPLOYMENT



Client / Edge Service Data store Async / Queue External API

○ ERASER DIAGRAM SPEC

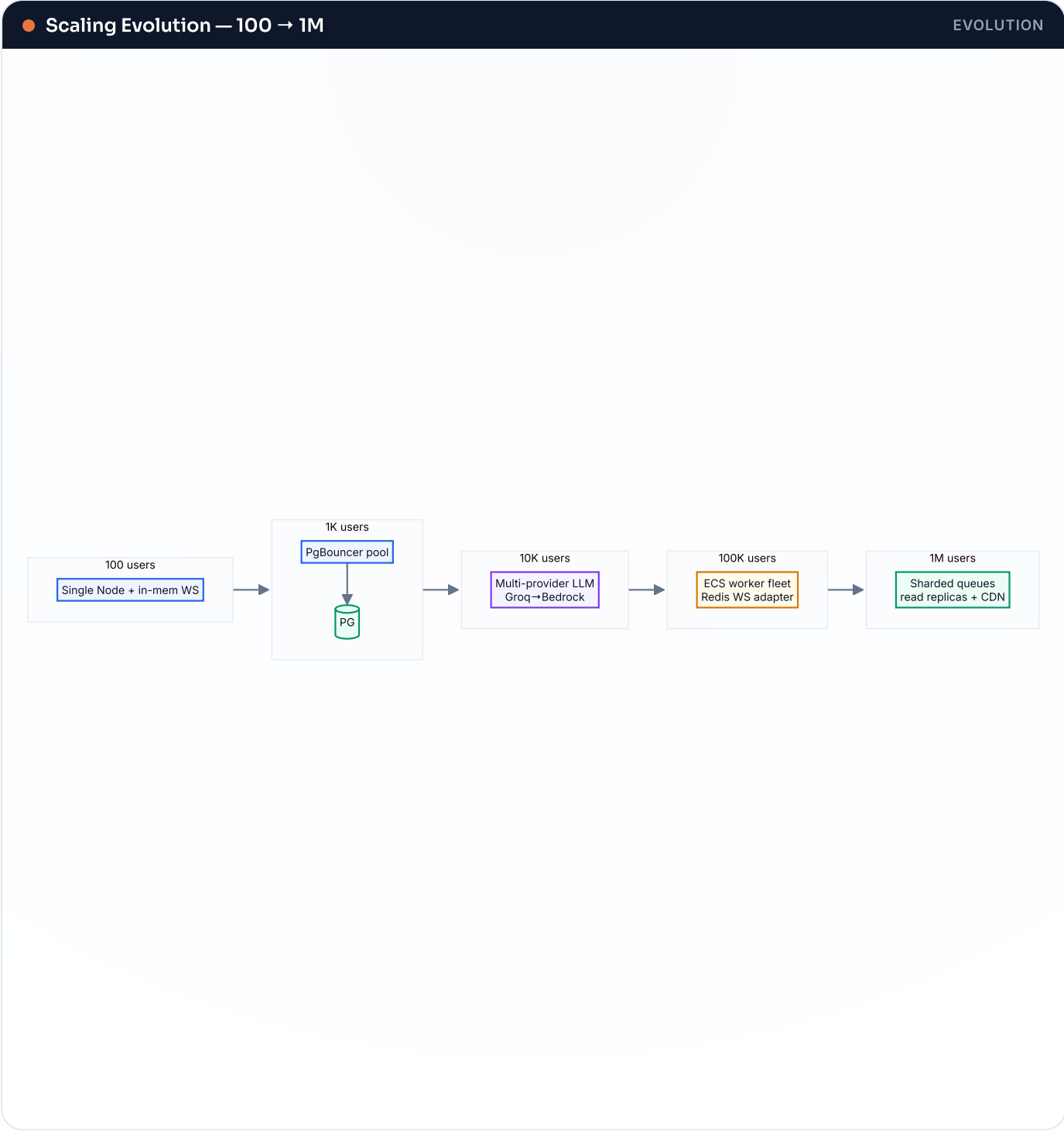
Users > CDN > Frontend > Edge auth > ALB
ALB > Gateway tasks
Gateway > Postgres + Redis
Redis > Agent workers
Agent > Postgres + LLM + S3
// zones: Vercel | AWS ECS | Managed Data

■ COMPONENT BREAKDOWN

Vercel edge	Static + SSR frontend, auth at the edge.
ECS Fargate	Serverless containers; gateway and agents scale independently.
PgBouncer	Connection pooling so Fargate scale-out doesn't exhaust PG.
Managed data	Supabase + Upstash + S3 — no self-hosted state.

■ VISUAL LAYOUT DESCRIPTION

Three labeled cloud boundaries stacked vertically: **Vercel** (edge/frontend, orange), **AWS ECS** (compute, blue), **Managed Data** (green/amber). Users enter top through the CDN. The key reviewer takeaway is the **independent scale groups 'xN'** on gateway and agent tasks, and that all state lives in the managed band — compute is stateless and disposable.



Scale	First Bottleneck	Architectural Move
100	Canvas state-sync lag	In-memory WS on a single node
1K	PG pool exhaustion	PgBouncer connection pooling
10K	LLM TPM/RPM limits	Multi-provider fallback (Groq→Bedrock)
100K	WS can't span instances	Redis Socket.io adapter + ECS fleet
1M	Queue backlog & read load	Sharded queues + PG read replicas + CDN

Security Architecture

Threat model & controls

14

The threat surface is a system that **executes model-generated code**. The core control is simple: that code never touches our infrastructure — it runs only in the user's browser sandbox.

Threat (STRIDE)	Vector	Mitigation
Tampering	Prompt forces dangerous shell commands	Code runs only in browser WebContainer — never on host
Info Disclosure	Secrets leak via generated output	Env vars AES-256 encrypted, injected via Vault at runtime
Elevation	Cross-tenant project access	Prisma global tenant filter + Clerk JWT
DoS	Abusive build spamming LLM	Per-user rate limits + queue depth caps



Never execute model output server-side — All generated code is sandboxed in-browser; the host never runs untrusted code, eliminating RCE as a class of bug.

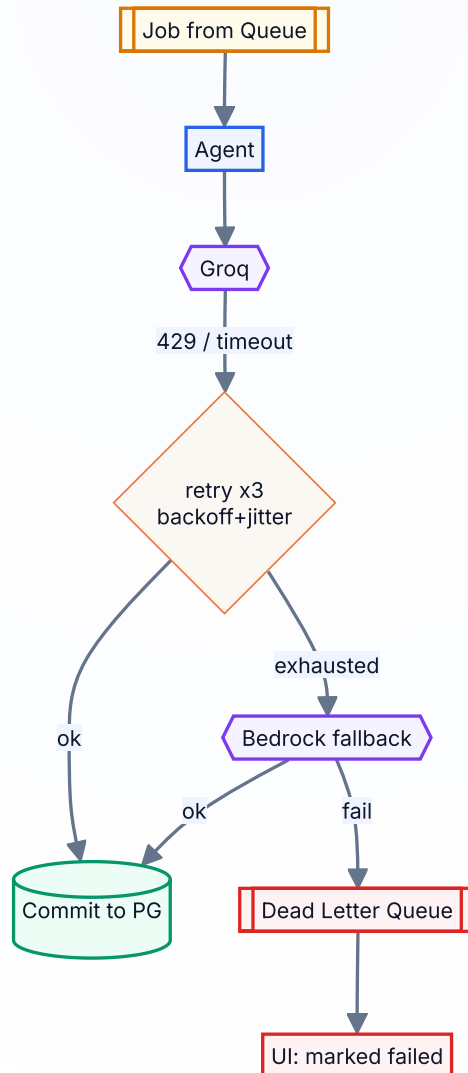
Failure Recovery

Diagram 7 — Failure & Recovery Topology

15

● Failure & Recovery Topology

RESILIENCE



Healthy Failed / Degraded Recovery path

○ ERASER DIAGRAM SPEC

Queue > Agent > Groq
Groq fail > retry(3, backoff+jitter)
retry ok > commit PG
retry exhausted > Bedrock fallback
Bedrock ok > commit
Bedrock fail > DLQ > UI failed

■ COMPONENT BREAKDOWN

Backoff+jitter	$2^n + \text{random ms}$ avoids thundering-herd retries.
Provider fallback	Groq exhausted → Bedrock, not instant failure.
DLQ	3x failure parks the job for inspection, not silent loss.
Honest UI	User always sees terminal state — failed, never spinning forever.

■ VISUAL LAYOUT DESCRIPTION

A vertical decision tree. The **happy path** (green) runs straight down the center. Failure branches peel off to the right into a red **retry** diamond, then a violet **fallback** provider, and finally the red **DLQ** at bottom-right. Color does the work: green = recovered, red = degraded, so the recovery story is legible at a glance.

Monitoring & Observability

Golden signals + alert matrix

16

Instrumented around the four golden signals. Every alert is actionable and maps to a runbook.



Latency
TTFT + node gen time (OTel spans).



Traffic
Builds/min, WS connections.



Errors
Gen failures, DLQ depth.



Saturation
Queue backlog, PG pool usage.

ALERTING MATRIX

Signal	Threshold	Action / Runbook
DLQ depth > 0	any failed job	Page on-call; inspect prompt/repair logs
Queue backlog > 100	sustained 5m	Scale agent workers on ECS
PG pool > 90%	sustained 2m	Raise PgBouncer pool / add replica
TTFT p99 > 200ms	sustained	Flip primary LLM provider

Cost Optimization

Unit economics under load

17

LLM cost scales with usage, not seats — so the whole strategy is to **not call the model** when we don't have to, and to call the cheapest model that's good enough when we do.

-70%

LLM SPEND

8B/70B

MODEL ROUTING

Cache

SKIP UNCHANGED NODES

LEVERS PULLED

- ✓ Prompt caching on static system templates
- ✓ Route layout/analysis to Llama-3 8B, only code to 70B
- ✓ Redis cache of compiled nodes — skip LLM if inputs unchanged
- ✓ Aggressive context pruning to cut tokens/call

Engineering Challenges

What broke, why, and what we learned

18

PRODUCTION INCIDENT — POST-MORTEM



The Incident

The Circular-Loop Hang. A user wired Node $A \rightarrow B$ and $B \rightarrow A$. The scheduler spun in an infinite loop, leaking memory and crashing Node.js gateways.



Root Cause

Kahn's algorithm assumed a strict DAG. With a cycle, no node ever reached in-degree 0, so the while-loop never terminated.



The Fix

Added a pre-save **Tarjan SCC** check that rejects cycle-creating edges, plus a hard iteration cap as a backstop.



Lasting Lesson

Never trust client validation. The canvas blocked loops in the UI, but the API must independently enforce graph integrity.



Blast radius — All gateway instances shared the bug; one malformed graph could OOM the fleet. Now a single poisoned input is rejected at the edge.

Architecture Tradeoffs

Chosen sacrifices, stated plainly

19

Every architecture is a set of deliberate sacrifices. Here are the ones we made on purpose — and what we gave up.

Decision	We Chose	We Sacrificed
Codegen unit	Per-node generation	Single-shot speed — many small calls vs one big one
Graph storage	Relational rows	JSON simplicity — but FK integrity is worth it
Sandbox	Browser WebContainer	Native perf — but zero server RCE risk
Realtime	WebSockets	Stateless simplicity — needs Redis adapter to scale

WHERE WE OPTIMIZED



Future Roadmap

Where the architecture goes next

20

The architecture is built to absorb two upgrades without a rewrite: collaborative editing and a self-healing compile loop.

V2**Real-time Collaboration**

Yjs (CRDT) for Google-Docs-style multi-user canvas editing with conflict-free merge.

V3**Self-Correction Sandbox**

Compiler catches TS/syntax errors and feeds them back into the LLM prompt for automatic repair.

V4**Template Marketplace**

Shareable graph templates; cache hot subgraphs across users to drop LLM spend further.

Interview Deep Dive

The questions that decide the loop

21

Q How do you handle LLM context limits across a multi-file system?

DAG decomposition. Instead of one mega-prompt, we generate node-by-node. Each child prompt receives only its parents' TypeScript interface signatures as static context, keeping active tokens under 3K and eliminating truncation.

[DAG](#)[Context pruning](#)[Type injection](#)

Q Why WebSockets over polling for generation updates?

Generation is push-shaped: the agent finishes a node at an unpredictable time. WS lets the server emit instantly, avoiding wasted DB read cycles from client polling. The cost is statefulness — solved with a Redis adapter when we scaled past one instance.

[WebSockets](#)[Redis adapter](#)[Push vs poll](#)

Q Explain Kahn's algorithm and where it runs here.

Topological sort by in-degree. Count each node's dependencies; nodes with 0 go first; as they complete, decrement children and enqueue any that hit 0. It produces the safe generation order so a referenced type always exists before its consumer.

[Topological sort](#)[In-degree](#)[Scheduler](#)

Q What was the hardest scaling bug?

WebSockets don't scale horizontally for free. A second container meant clients on different nodes couldn't receive each other's events. Fixed with the Socket.io Redis adapter so broadcasts fan out across the whole fleet.

[Horizontal scale](#)[Pub/Sub](#)[Sticky state](#)

Revision Sheet

One-page recall before the round

22

The 60-second mental model: **codebase = DAG**; sort it, generate per-node with parent types injected, sandbox the preview.

ARCHITECTURE

ReactFlow canvas → Node gateway → Redis queue → FastAPI agent → Llama-3 → Postgres. Async return via WS.

CORE ALGORITHM

Kahn topo-sort for build order; Tarjan SCC to reject cycles before persistence.

DATABASE

Project 1—N Nodes & Edges. Edges are relational rows; (project_id,status) composite index drives the scheduler.

SCALING

PgBouncer → multi-provider LLM → Redis WS adapter → sharded queues + replicas.

SECURITY

Browser-only WebContainer (no server RCE), Vault-injected secrets, Prisma tenant filter.

SIGNATURE BUG

Circular-dependency OOM → fixed with pre-save Tarjan check + iteration cap.



60-second pitch — CraftaStudio turns 'generate my app' into a graph-scheduling problem: model dependencies as a DAG, topologically sort it, and generate each node with its parents' types injected — so multi-file output actually compiles. It's async end-to-end, sandboxes previews in the browser, and falls back across LLM providers for 99.9% generation success.

PROJECT DOSSIER · 02

PlacePro

Campus placement management with a real-time event bus and database-enforced row-level security.

EdTech

DOMAIN

Redis Pub/Sub

CORE

10K

CONCURRENT

Tier 1

PRIORITY

Executive Summary

What it is, who it serves, why it matters

01

PlacePro is a campus placement portal engineered for the brutal traffic shape of placement season: **10K students hammering registration in a 5-minute window**. It survives with a stateless app tier, an event bus, and DB-enforced security.

10K

CONCURRENT

<1ms

PUB/SUB

0

LOST APPS

RLS

DB-LEVEL AUTHZ



The Core Problem

Placement portals crash under the concurrent load of short registration windows, and weak API routing lets students tamper with other candidates' application records.



Target Users & Value

University coordinators and students. Streamlines applications, eligibility screening, and recruiter updates in real time.



Business Goal & Impact

Support **10,000 concurrent registrations** with zero dropped applications and strict, compliance-grade data isolation.



My Role & Ownership

System Architect & Backend Engineer. Implemented PostgreSQL Row-Level Security policies and the Redis Pub/Sub real-time engine.

Product Vision

The bet, the wedge, the moat

02

Legacy portals are CRUD apps that buckle under burst load and trust the application layer for security. PlacePro is **event-driven and database-secured** — load is absorbed by stateless workers, and authorization is non-bypassable.

ALTERNATIVES WE REJECTED

Alternative	Where It Breaks	PlacePro Edge
Google Sheets	Concurrent-edit conflicts; zero access control.	RLS + atomic transactions.
Legacy ERP	Slow UI; manual eligibility review.	Automated instant eligibility engine.
WS everywhere	Stateful, heavy for one-way status.	SSE for lightweight server push.

CORE CAPABILITY MATRIX



Auto-Eligibility

Blocks apps failing CGPA/backlog caps.



Live Status

SSE updates on recruitment progress.



Role Isolation

Students, recruiters, admins fully partitioned.

PLACEPRO · SECTION 03

Interview Yield ★★★★★

Business Problem

Quantifying the pain we remove

03

A placement portal that drops applications during the registration rush isn't a bug — it's a **career-altering failure** for a student who missed a one-shot window. Reliability under burst is the entire product.



Burst Collapse

5-minute windows where 10K students arrive at once.



Tamper Risk

Weak routing lets students edit others' records.



Stale Status

Students refresh endlessly for shortlist updates.



Manual Review

Coordinators hand-check eligibility for hundreds.



Why now — Managed Postgres with native RLS (Supabase) makes database-enforced multi-tenancy practical without a custom auth layer.

COST OF THE STATUS QUO



User Journey

From first touch to recurring value

04

A student goes from profile to confirmed application — with live status — without ever seeing a spinner or a 500.

- 1 Onboard**
Student verifies email, fills CGPA/backlog profile once.
- 2 Browse**
Only eligible job postings are shown; ineligible ones are greyed with the reason.
- 3 Apply**
One click; eligibility re-checked server-side, application committed under RLS.
- 4 Track**
SSE pushes shortlist/interview/offer status live — no refresh needed.
- 5 Coordinate**
Admins watch a real-time dashboard of applications as they land.

Functional Requirements

P0 / P1 capability contract

05

Scoped to the critical path. Each requirement maps to an owned subsystem and an acceptance signal.

MUST-HAVE (P0)

- ✓ Eligibility-gated applications (CGPA/backlog)
- ✓ Row-level data isolation per student
- ✓ Real-time status broadcast (SSE)
- ✓ Role-based dashboards (student/recruiter/admin)
- ✓ Burst-tolerant write path

SHOULD-HAVE (P1)

- › Resume upload to object storage
- › Bulk shortlist by recruiters
- › Email digests
- › Audit log of status changes

ID	Capability	Acceptance Signal
FR-1	Eligibility gate	Ineligible apply blocked server-side
FR-2	Isolation	Student cannot read others' rows
FR-3	Live status	Status change pushed < 1s
FR-4	Burst	10K applies in 5 min, 0 dropped

Non-Functional Requirements

SLOs, not vibes

06

The quality attributes that define production-readiness — each with a measurable target and the mechanism that enforces it.

Attribute	Target (SLO)	Enforcement Mechanism
Availability	99.95% during drives	Stateless workers + autoscale + WAF
Throughput	10K writes / 5 min	Async FastAPI + connection pooling
Latency	Status push < 1s	Redis Pub/Sub + SSE
Security	Non-bypassable isolation	Postgres RLS policies
Durability	Zero lost applications	Transactional writes + outbox for events

QUALITY ATTRIBUTE PRIORITY



Tech Stack Decisions

Every choice has a bill

07

Chosen for burst tolerance and security guarantees: async Python for I/O-bound load, managed Postgres for RLS, Redis for the cheapest possible fan-out.

Layer	Chosen	Why This	Trade-off Accepted
Frontend	Next.js · Tailwind	SSR validation + fast dashboards.	Client bundle weight.
Backend	FastAPI	Async throughput + auto OpenAPI docs.	Fewer middleware libs than Express.
Database	PostgreSQL (Supabase)	Relational integrity + native RLS.	Connection ceiling under burst.
Bus	Redis Pub/Sub	Sub-ms broadcast for live status.	In-memory; needs outbox for durability.
Infra	Docker · AWS ECS	Containerized autoscale for drives.	Task-def config overhead.

PLACEPRO · SECTION 08

Interview Yield ★★★★★

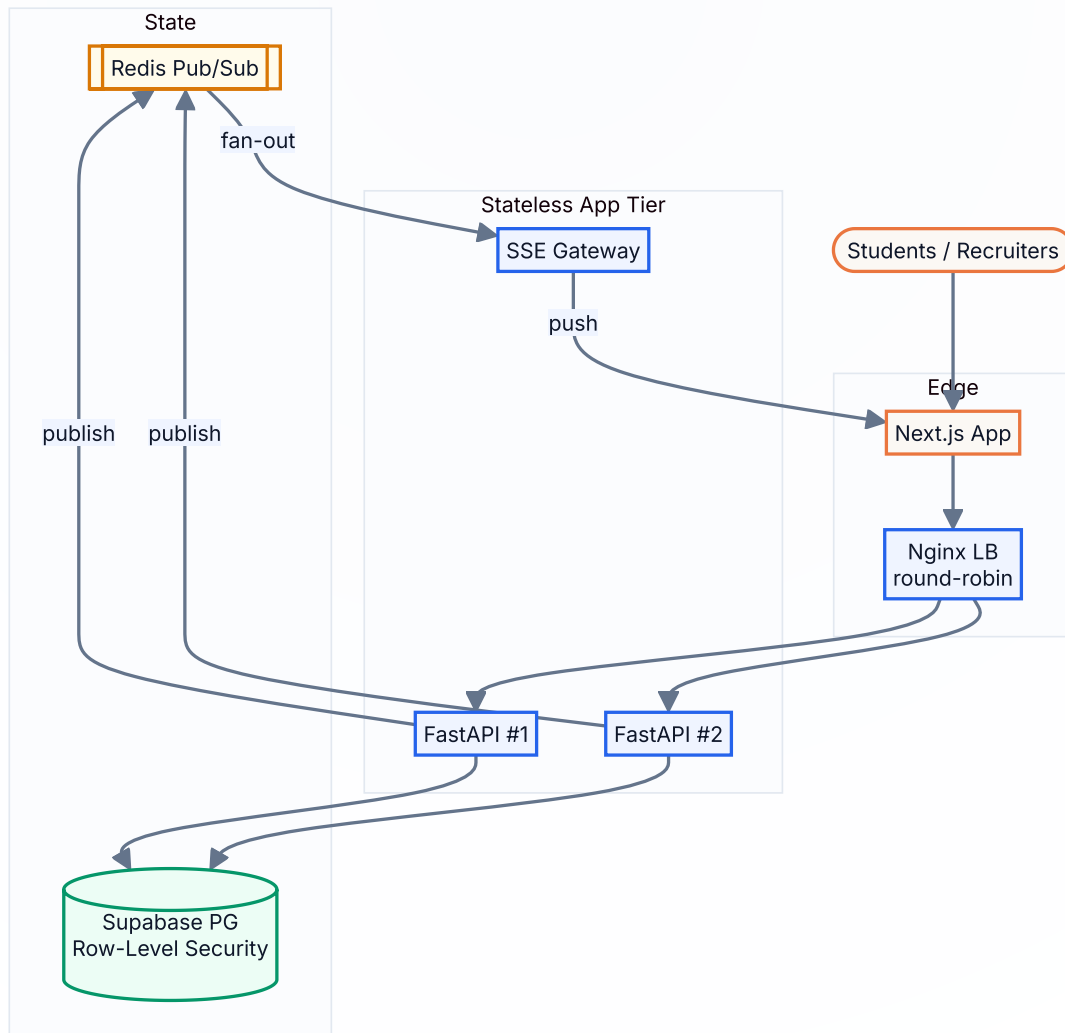
Architecture Overview

Diagram 1 — System Architecture

08

● PlacePro — System Architecture

C4 / CONTAINER



○ ERASER DIAGRAM SPEC

```
Students [icon: users]
Frontend [icon: nextjs, color: orange]
LB [icon: nginx, color: blue]
FastAPI [icon: python, color: blue]
SSE [icon: broadcast, color: blue]
Postgres [icon: postgresql, color: green]
Redis [icon: redis, color: amber]
Students > Frontend > LB > FastAPI
FastAPI > Postgres
FastAPI > Redis: publish
Redis > SSE > Frontend: push
```

▣ COMPONENT BREAKDOWN

- Nginx LB** Round-robins across stateless FastAPI workers.
- FastAPI tier** Async business logic; horizontally scalable, no local state.
- RLS Postgres** Tenant isolation enforced in the query planner, not app code.
- Redis Pub/Sub** Sub-ms event broadcast; SSE pushes live status to clients.

▣ VISUAL LAYOUT DESCRIPTION

Top-down with three bands. Users enter through the orange **edge** (frontend + Nginx). The **app tier** shows two identical FastAPI boxes to signal statelessness/horizontal scale. The **state** band pairs green Postgres with the amber Redis bus. The signature loop is the **publish** → **fan-out** → **push** path on the right, proving updates are event-driven, not polled.

PLACEPRO · SECTION 09

Interview Yield ★★★★★

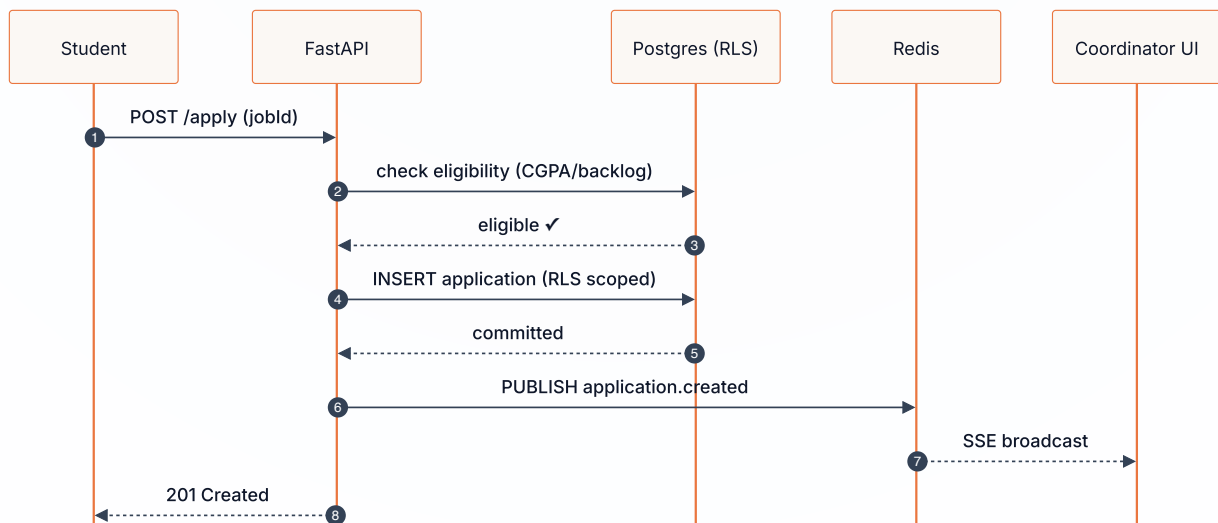
Request Lifecycle

Diagram 2 — End-to-End Request Flow

09

● Apply to Job — Request Lifecycle

SEQUENCE



○ ERASER DIAGRAM SPEC

```
Student > FastAPI: POST /apply
FastAPI > Postgres: eligibility check
Postgres > FastAPI: eligible
FastAPI > Postgres: INSERT (RLS)
FastAPI > Redis: PUBLISH event
Redis > Coordinator: SSE
FastAPI > Student: 201
```

■ COMPONENT BREAKDOWN

Eligibility gate	CGPA/backlog rules checked before any write.
RLS insert	Row stamped with tenant; cross-student writes impossible.
Publish	Decouples the write from notifying coordinators.
SSE	One-way server push — lighter than WS for status feeds.

▣ VISUAL LAYOUT DESCRIPTION

Five-actor sequence. The **eligibility check** round-trip sits up top as a gate before the write. The **PUBLISH** → **SSE** pair branches sideways to the Coordinator lifeline, visually separating the user's response path (201 back to Student) from the broadcast path — two independent outcomes from one request.

PLACEPRO · SECTION 10

Interview Yield ★★★★★

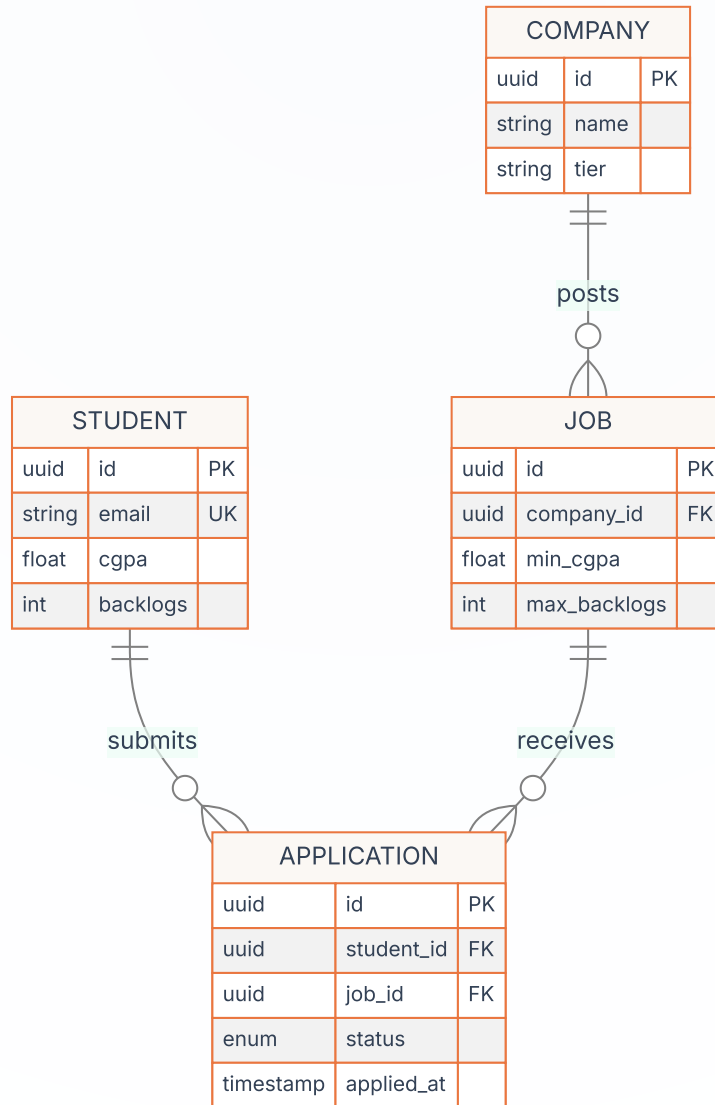
Database Design

Diagram 3 — Entity Relationship Model

10

PlacePro — Data Model

ERD



ERASER DIAGRAM SPEC

```
STUDENT { id pk; email uk; cgpa; backlogs }
COMPANY { id pk; name; tier }
JOB { id pk; company_id fk; min_cgpa; max_backlogs }
APPLICATION { id pk; student_id fk; job_id fk; status }
STUDENT 1-* APPLICATION
COMPANY 1-* JOB
JOB 1-* APPLICATION
```

COMPONENT BREAKDOWN

- Eligibility cols** min_cgpa/max_backlogs on JOB drive the gate.
- Junction** APPLICATION is the M:N bridge between STUDENT and JOB.
- Status enum** applied → shortlisted → offered, indexed for dashboards.
- Unique email** Prevents duplicate student identities.

VISUAL LAYOUT DESCRIPTION

Four entities in a diamond: **STUDENT** and **COMPANY** on the outer flanks, both fanning into the central **APPLICATION** junction (via **JOB**). The crow's-feet make the many-to-many between students and jobs obvious. Eligibility columns are highlighted on JOB to show where the business rule lives in the schema.

PLACEPRO · SECTION 11

Interview Yield ★★★★★

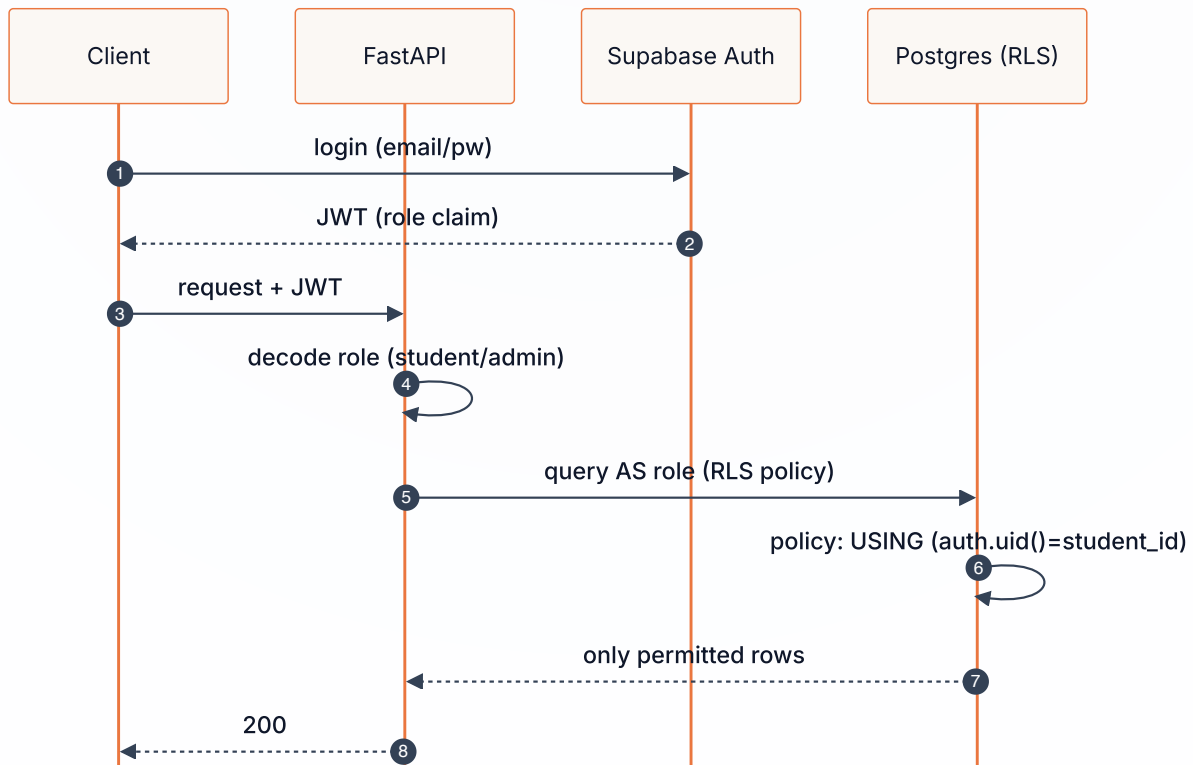
Authentication Design

Diagram 4 — Identity & Authorization Flow

11

● RBAC + Row-Level Security

SEQUENCE



○ ERASER DIAGRAM SPEC

```
Client > Supabase Auth: login
Auth > Client: JWT role
Client > FastAPI: JWT
FastAPI > Postgres: query as role
Postgres: RLS policy filter
Postgres > FastAPI: scoped rows
FastAPI > Client: 200
```

■ COMPONENT BREAKDOWN

Supabase Auth Issues JWT carrying the role claim.**Role decode** App routes by role; coordinators see all, students see self.**RLS policy** USING(auth.uid())=student_id enforced by the DB engine.**Two-layer** App authz + DB RLS — a bug in one is caught by the other.

■ VISUAL LAYOUT DESCRIPTION

Four lifelines. The lower half is dominated by the **RLS policy** self-message on the Postgres lifeline — the diagram's thesis that authorization lives *in the database*. A note could read 'auth.uid() = student_id'. The role-decode self-message on FastAPI marks the app-layer check that complements it.

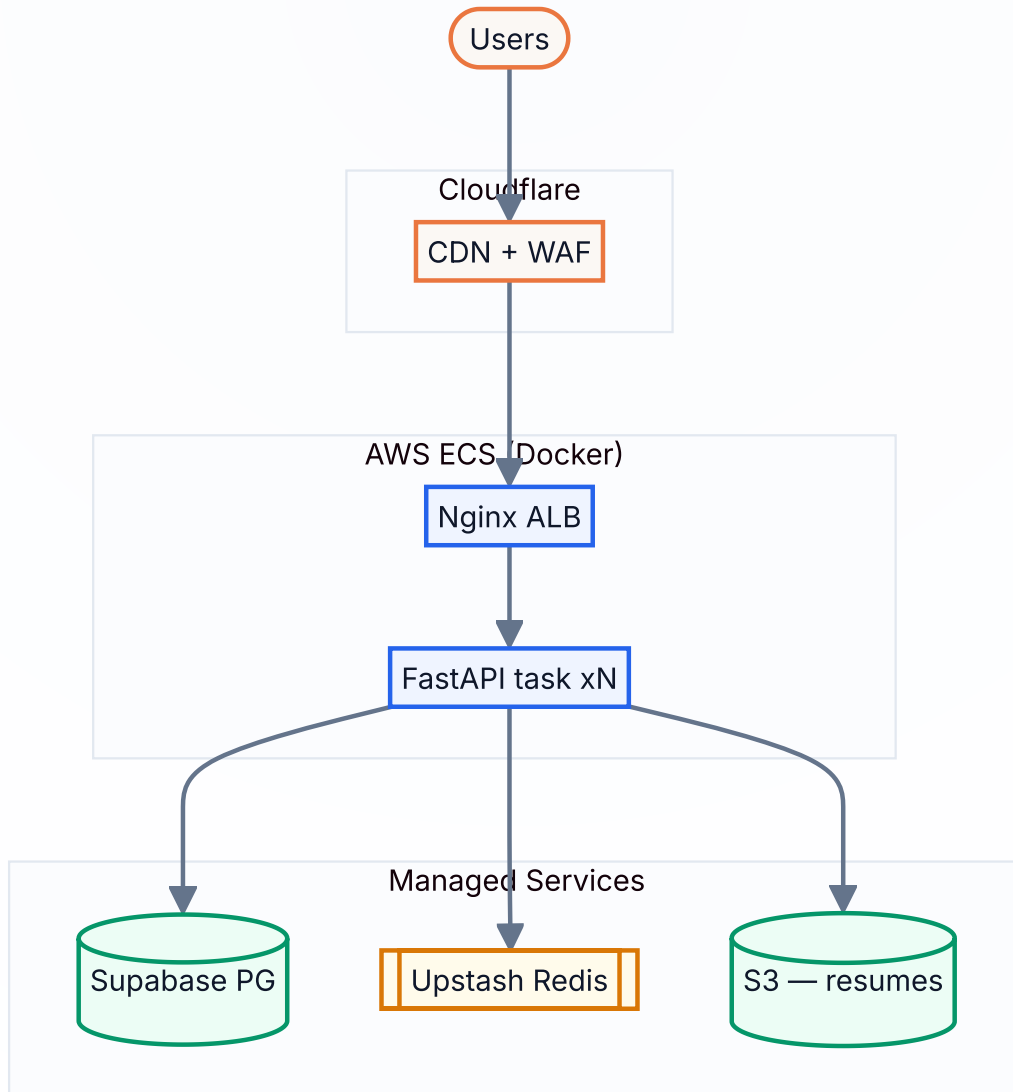
Deployment Architecture

Diagram 5 — Cloud Deployment Topology

12

● Cloud Deployment Topology

DEPLOYMENT



Client / Edge Service Data store Async / Queue External API

○ ERASER DIAGRAM SPEC

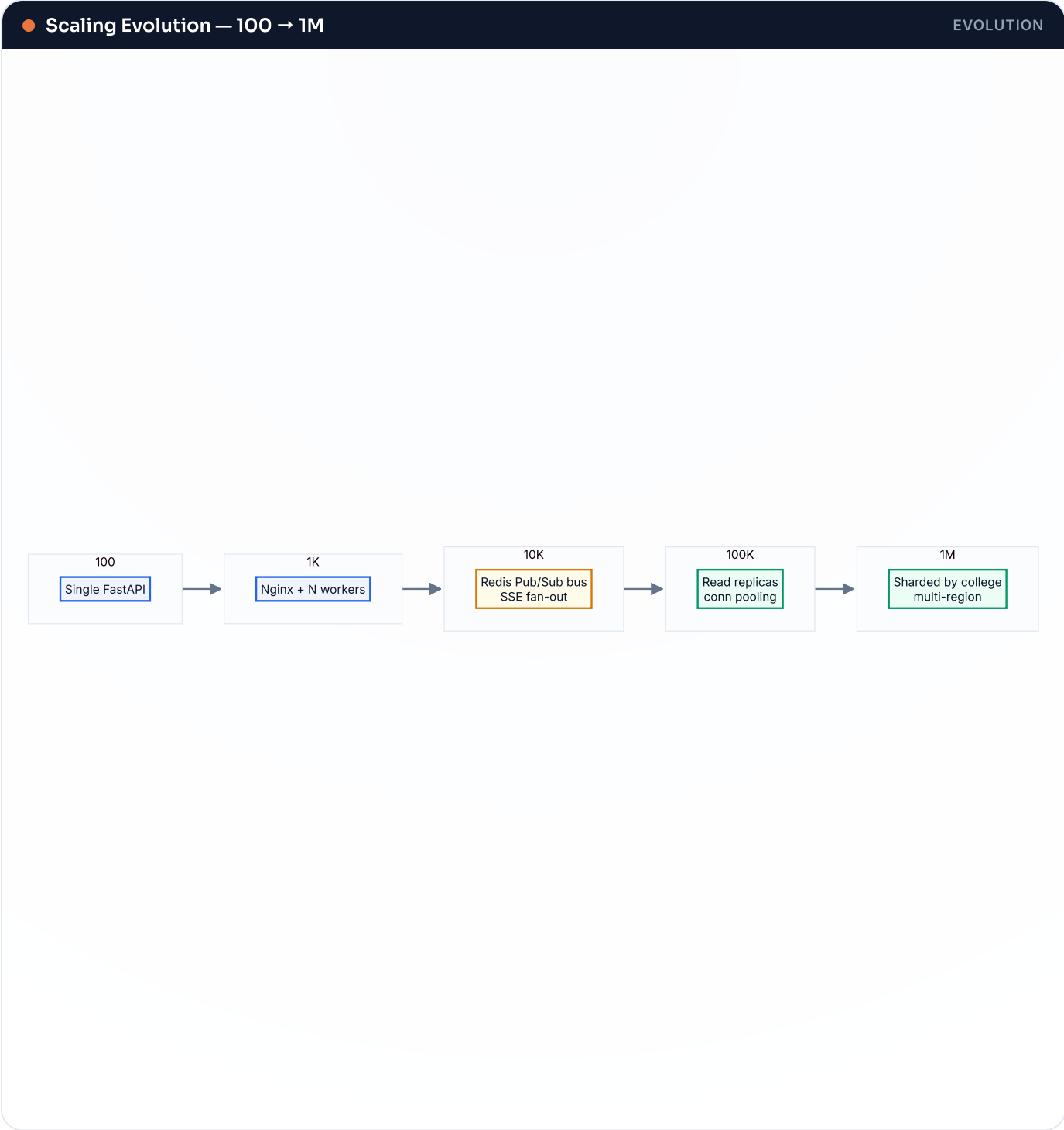
Users > CDN+WAF > Nginx ALB > FastAPI tasks
FastAPI > Postgres + Redis + S3
// zones: Cloudflare | AWS ECS | Managed

■ COMPONENT BREAKDOWN

Cloudflare	CDN + WAF absorbs spikes and filters attacks at the edge.
ECS autoscale	FastAPI tasks scale on CPU/queue during placement drives.
Managed state	Supabase + Upstash + S3 — fully managed, no ops toil.
S3 resumes	Large binary artifacts off the relational DB.

■ VISUAL LAYOUT DESCRIPTION

Three stacked cloud zones. The reviewer's eye should land on the **xN** on FastAPI tasks (burst scale for registration windows) and the WAF at the edge (the portal's history of being DDoSed during drives). All durable state is in the managed band — ECS tasks are cattle, not pets.



Scale	First Bottleneck	Architectural Move
100	Single worker CPU	One FastAPI process
1K	Request concurrency	Nginx round-robin + worker pool
10K	DB write hot-spot	Pub/Sub + SSE offloads status reads
100K	Dashboard read load	Read replicas + PgBouncer
1M	Single-DB ceiling	Shard by college + multi-region

Security Architecture

Threat model & controls

14

The crown-jewel control is **Row-Level Security**: even if an attacker reaches the database with a valid session, the query planner physically cannot return another student's rows.

Threat (STRIDE)	Vector	Mitigation
Elevation	Student edits another's application	Postgres RLS USING(auth.uid())=student_id)
Spoofing	Forged role to access admin views	Signed Supabase JWT with verified role claim
DoS	Registration-window flood	Cloudflare WAF + rate limit + autoscale
Repudiation	Disputed status changes	Append-only audit log



Authorization belongs in the database — App-layer checks can be bypassed by any routing bug; RLS makes cross-tenant reads impossible at the engine level.

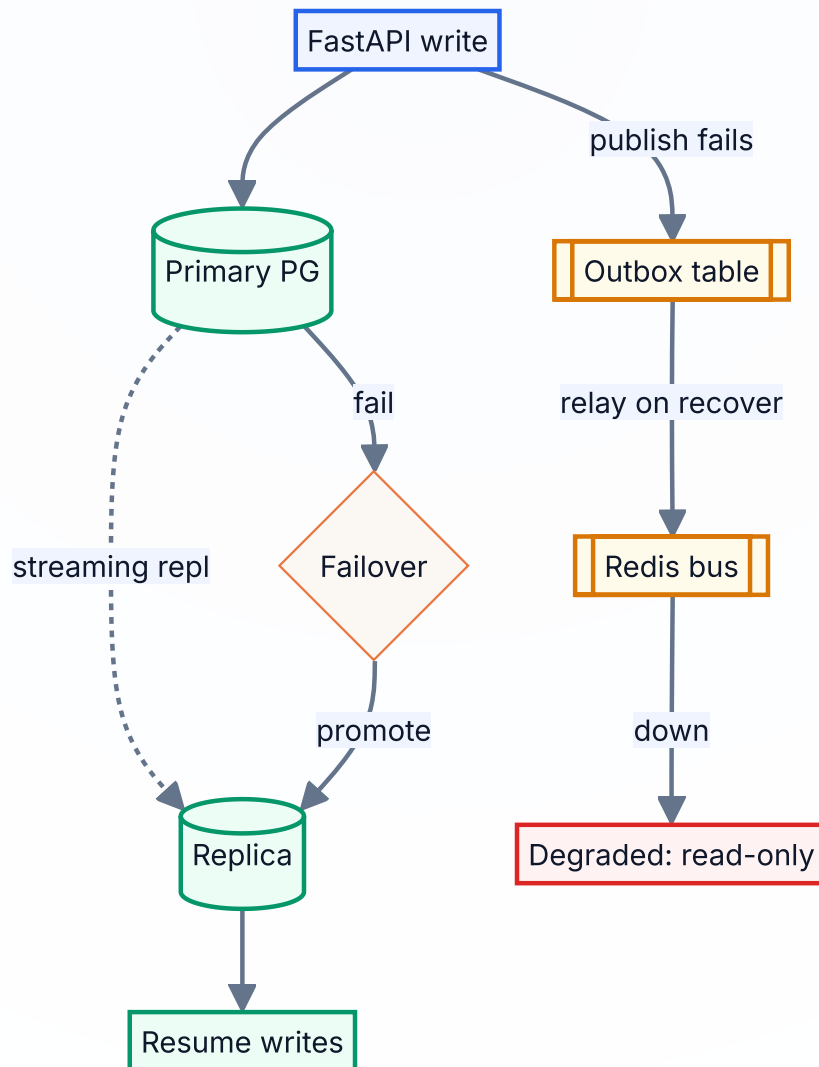
Failure Recovery

Diagram 7 — Failure & Recovery Topology

15

● Failure & Recovery Topology

RESILIENCE



● Healthy ● Failed / Degraded ● Recovery path

○ ERASER DIAGRAM SPEC

Write > Primary PG
Primary > Replica: streaming repl
Primary fail > Failover > promote Replica > resume
Write > Outbox (if publish fails)
Outbox > Redis on recover
Redis down > degraded read-only

■ COMPONENT BREAKDOWN

Streaming repl Replica stays warm; promotion on primary loss.

Outbox pattern If Redis publish fails, event is persisted and relayed later — no lost notifications.

Graceful degrade Redis down → reads still served, writes queue.

Failover Promote replica; brief write pause, no data loss.

▣ VISUAL LAYOUT DESCRIPTION

Two failure stories side by side. Left: **DB failover** — primary (green) streams to replica; on failure a diamond promotes the replica. Right: **event-bus failure** — the Outbox (amber) captures events when Redis is down and relays on recovery. Red nodes mark degraded states; the eye reads green→red→green as 'recover'.

PLACEPRO · SECTION 16

Interview Yield ★★★★★

Monitoring & Observability

Golden signals + alert matrix

16

Instrumented around the four golden signals. Every alert is actionable and maps to a runbook.

**Latency**

Apply p99, SSE delivery time.

**Traffic**

Applies/min, active SSE conns.

**Errors**

5xx rate, failed publishes.

**Saturation**

PG connections, worker CPU.

ALERTING MATRIX

Signal	Threshold	Action / Runbook
5xx > 1%	sustained 1m	Scale ECS tasks; check PG pool
PG conns > 90%	2m	Add replica / raise PgBouncer
Publish failures > 0	any	Verify outbox relay draining
SSE delivery > 2s	sustained	Inspect Redis latency

PLACEPRO · SECTION 17

Interview Yield ★★★★★

Cost Optimization

Unit economics under load

17

Drives are spiky: massive load for hours, near-idle the rest of the term. The whole cost play is **scale-to-zero between drives** and never paying for idle capacity.

-60%

IDLE SPEND

Autoscale

0→N ON DEMAND

Managed

NO DB OPS COST

LEVERS PULLED

- ✓ ECS scale-to-near-zero between drives
- ✓ Serverless Redis (pay-per-request) via Upstash
- ✓ Resumes in S3 (cheap) not in the DB
- ✓ Read replicas only spun up for active drives

Engineering Challenges

What broke, why, and what we learned

18

PRODUCTION INCIDENT — POST-MORTEM



The Incident

The Registration Stampede. A marquee company opened applications; 8K students hit POST /apply within 90 seconds and the DB connection pool exhausted, returning 500s.



Root Cause

Each FastAPI worker opened its own DB connections with no pooling. Under burst, connections blew past Postgres's limit and new requests were refused.



The Fix

Introduced **PgBouncer** transaction pooling and moved status reads onto the Redis/SSE path so the DB only handled writes during the spike.



Lasting Lesson

Connection management is a first-class scaling concern. The bottleneck wasn't CPU — it was a finite resource (connections) nobody had pooled.



Blast radius — Every student hitting the portal during the window saw 500s for ~4 minutes — a one-shot application window nearly missed. Pooling cut peak connections 10x.

Architecture Tradeoffs

Chosen sacrifices, stated plainly

19

Every architecture is a set of deliberate sacrifices. Here are the ones we made on purpose — and what we gave up.

Decision	We Chose	We Sacrificed
Real-time	SSE (one-way)	WS bidirectionality — but SSE is lighter for status feeds
Security	DB-level RLS	App simplicity — but RLS is non-bypassable
Bus	Redis Pub/Sub	Durability — added an outbox to compensate
Scale	Stateless workers	Sticky sessions — kept all state in PG/Redis

WHERE WE OPTIMIZED



Future Roadmap

Where the architecture goes next

20

Next moves harden durability and widen the real-time surface without disturbing the stateless core.

V2**Durable Event Log**

Swap raw Pub/Sub for Redis Streams / Kafka so notifications survive a bus restart natively.

V3**Recruiter Workflows**

Multi-stage pipelines (OA → interview → offer) with automated transitions and SLA timers.

V4**Analytics**

Placement funnel dashboards; per-company conversion and offer-acceptance analytics.

Interview Deep Dive

The questions that decide the loop

21

Q How does Row-Level Security stop a student reading another's data?

RLS attaches a policy to the table — e.g. `USING (auth.uid() = student_id)`. The Postgres planner appends this predicate to **every** query automatically. Even a SQL-injection or a buggy endpoint can't return rows the policy forbids, because filtering happens in the engine, below the application.

RLS

Defense in depth

Multi-tenancy

Q Why SSE instead of WebSockets for live status?

Status updates are **one-directional** (server→client). SSE rides plain HTTP, auto-reconnects, and is far lighter to operate than full-duplex WS. We reserve WS for genuinely bidirectional features; for a status feed, SSE is the right tool.

SSE

WebSockets

Push model

Q How did you survive the registration stampede?

Three moves: **PgBouncer** transaction pooling so 10K requests share a small connection set; stateless FastAPI workers autoscaling on ECS; and offloading status reads to Redis/SSE so the database during the spike only handled the critical writes.

Connection pooling

Autoscale

Read offload

Q Redis Pub/Sub isn't durable — how do you avoid lost notifications?

An **outbox pattern**: the event is written in the same DB transaction as the application, and a relay publishes it to Redis. If Redis is down, the event waits in the outbox and is replayed on recovery — so a bus blip never loses a notification.

Outbox

Durability

At-least-once

Revision Sheet

One-page recall before the round

22

Mental model: **stateless workers + event bus + DB-enforced security**. Burst-tolerant by construction.

ARCHITECTURE

Next.js → Nginx → stateless FastAPI → Postgres(RLS); Redis Pub/Sub → SSE for live status.

SECURITY

Postgres RLS policies make cross-tenant reads impossible at the engine level; signed JWT roles.

DATABASE

STUDENT/COMPANY/JOB/APPLICATION; APPLICATION is the M:N junction; eligibility cols on JOB.

SCALING

Pool (PgBouncer) → bus (Redis) → replicas → shard-by-college. Compute is stateless cattle.

DURABILITY

Outbox pattern bridges non-durable Pub/Sub so notifications are never lost.

SIGNATURE BUG

Registration stampede exhausted the connection pool → fixed with PgBouncer + read offload.



60-second pitch — PlacePro is a placement portal built for the 5-minute stampede. Stateless FastAPI workers autoscale behind Nginx, Postgres Row-Level Security makes tenant isolation non-bypassable, and a Redis Pub/Sub bus with SSE pushes live status without hammering the DB. An outbox pattern guarantees no notification is ever lost.

03

PROJECT DOSSIER · 03

Mnemo

AI long-term memory engine with Ebbinghaus spaced decay and pgvector hybrid search.

AI Infra

DOMAIN

pgvector

CORE

-70%

TOKEN COST

Tier 1

PRIORITY

MNEMO · SECTION 01

Interview Yield ★★★★★

Executive Summary

What it is, who it serves, why it matters

01

Mnemo gives stateless LLM agents a **persistent, searchable memory**. It stores conversation facts as vectors, ranks recall by an Ebbinghaus forgetting curve, and returns only the top-k relevant blocks — cutting prompt token cost ~70%.

-70%

TOKEN COST

pgvector

ONE DATASTORE

Hybrid

VECTOR+BM25

top-k

RECALL SIZE



The Core Problem

AI agents have no native long-term memory: they suffer amnesia on reboot, and stuffing full history into the prompt causes context truncation and runaway token costs.



Target Users & Value

LLM agents and chat products needing contextual persistence. Mnemo cuts token spend by retrieving only relevant memory blocks.



Business Goal & Impact

Decrease LLM context token cost by **70%** while *improving* retrieval relevance by ranking memories on a decay score.



My Role & Ownership

Core Developer & System Designer. Designed the pgvector embedding schema, the Ebbinghaus decay engine, and the hybrid keyword/vector search.

Product Vision

The bet, the wedge, the moat

02

Most memory layers do naive vector similarity and resurface stale junk. Mnemo treats memory like a human brain: **relevance decays over time** unless reinforced by access — so recall stays fresh, not just similar.

ALTERNATIVES WE REJECTED

Alternative	Where It Breaks	Mnemo Edge
Full-context stuffing	Truncation + huge token bills.	Top-k retrieval, ~70% cheaper.
Pinecone / vector DB	Separate system to operate + sync.	pgvector — one Postgres for all of it.
Pure cosine recall	Resurfaces stale, irrelevant memories.	Decay-weighted re-ranking.

CORE CAPABILITY MATRIX



Spaced Decay

Ebbinghaus curve sinks unused memories.



Hybrid Search

Vector ANN + BM25 keyword in one query.



Top-k Recall

Returns a few blocks, not the whole log.

MNEMO · SECTION 03

Interview Yield ★★☆☆☆

Business Problem

Quantifying the pain we remove

03

Every token in the context window is paid for on **every** call. An agent that re-reads its whole history is burning money linearly with conversation length. Mnemo makes memory cost *sublinear*.



Token Burn

Full-history prompts cost **more every turn**.



Amnesia

Stateless agents forget everything on reboot.



Truncation

Long histories get silently cut, losing key facts.



Stale Recall

Naive similarity surfaces outdated memories.



Why now — pgvector matured into a production ANN index, so semantic memory no longer needs a separate, costly vector database.

COST OF THE STATUS QUO



User Journey

From first touch to recurring value

04

An agent stores facts as it learns them and recalls just what's relevant — staying coherent across sessions for pennies.

- 1 Remember**
Agent calls /remember with a fact; Mnemo embeds and stores it with importance + timestamp.
- 2 Live**
Each memory accrues access_count and last_access as it gets used.
- 3 Recall**
Agent calls /recall with a query; hybrid search finds candidates.
- 4 Re-rank**
Decay scorer weights candidates by relevance × freshness × importance.
- 5 Inject**
Only the top-k blocks enter the prompt — cheap, sharp context.

Scoped to the critical path. Each requirement maps to an owned subsystem and an acceptance signal.

- MUST-HAVE (P0)
- SHOULD-HAVE (P1)
- ✓ /remember and /recall API

✓ Embedding on write

✓ Hybrid vector + BM25 retrieval

✓ Ebbinghaus decay re-ranking

✓ Per-namespace tenant isolation

› Importance tagging

› Embedding cache

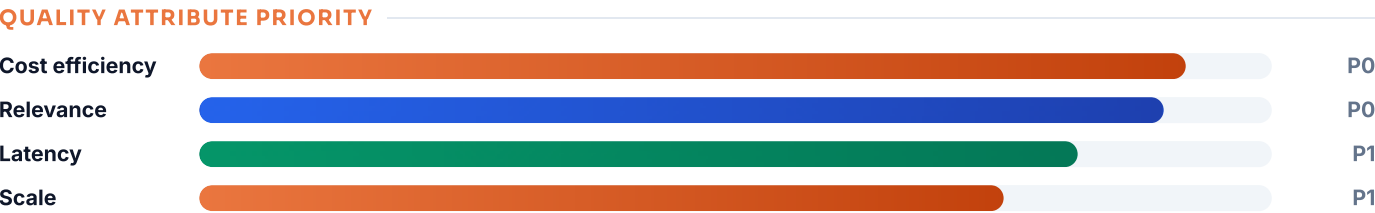
› Memory consolidation/merge

› TTL hard-expiry for sensitive data

ID	Capability	Acceptance Signal
FR-1	Store	/remember embeds + persists < 200ms
FR-2	Retrieve	Hybrid recall returns top-k
FR-3	Decay	Unused memories rank lower over time
FR-4	Isolation	Namespace scopes every query

The quality attributes that define production-readiness — each with a measurable target and the mechanism that enforces it.

Attribute	Target (SLO)	Enforcement Mechanism
Latency	Recall p99 < 150ms	HNSW index + embed cache
Cost	−70% vs full-context	Top-k retrieval, not full history
Relevance	Recall@k beats cosine baseline	Decay-weighted hybrid ranking
Scalability	1M memories / namespace	ANN index + sharding
Isolation	No cross-tenant recall	Namespace hard filter



Tech Stack Decisions

Every choice has a bill

07

Deliberately boring infra: one Postgres does relational + vector + keyword search, so there's a single system to back up, secure, and reason about.

Layer	Chosen	Why This	Trade-off Accepted
API	FastAPI · Python	Async + rich ML/embedding ecosystem.	GIL limits CPU-bound work.
Store	Postgres + pgvector	Vectors, rows, BM25 in one DB.	ANN tuning required at scale.
Embeddings	OpenAI text-embedding-3	Strong semantic quality.	External dependency + cost.
Cache	Redis	Caches query embeddings + hot recalls.	Extra component to run.
Ranking	Custom decay scorer	Encodes the forgetting curve.	Needs tuning per domain.

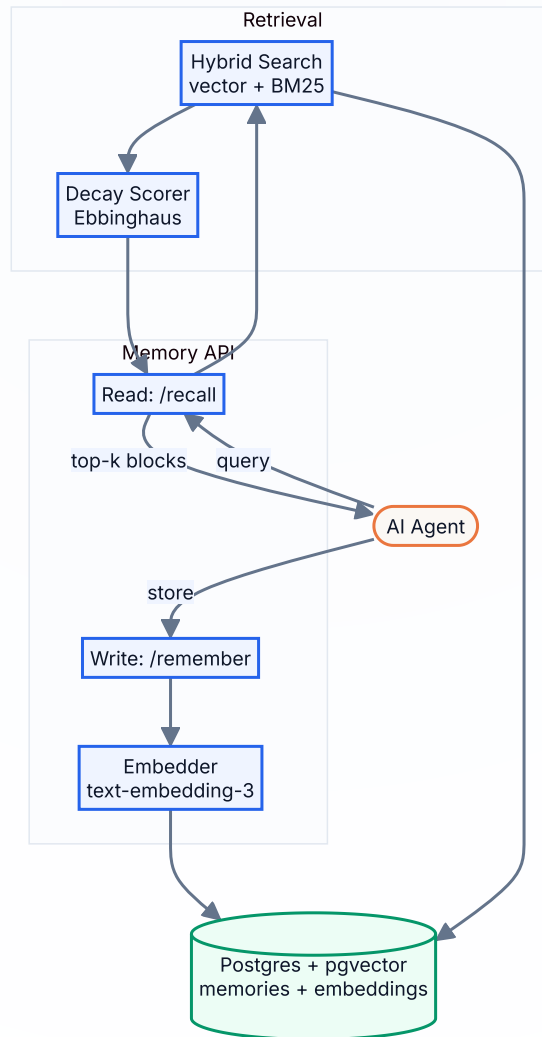
Architecture Overview

Diagram 1 — System Architecture

08

● Mnemo — System Architecture

C4 / CONTAINER



○ ERASER DIAGRAM SPEC

```
Agent [icon: robot]
Write [icon: save, color: blue]
Read [icon: search, color: blue]
Embedder [icon: vector, color: blue]
HybridSearch [icon: merge, color: blue]
DecayScorer [icon: clock, color: blue]
Postgres [icon: postgresql, color: green]
Agent > Write > Embedder > Postgres
Agent > Read > HybridSearch > Postgres
HybridSearch > DecayScorer > Read
Read > Agent: top-k
```

▣ COMPONENT BREAKDOWN

Embedder	Turns memory text into a vector on write.
Hybrid search	Combines pgvector ANN with BM25 keyword recall.
Decay scorer	Re-ranks by Ebbinghaus forgetting curve + recency.
pgvector	One Postgres stores rows, embeddings, and the ANN index.

▣ VISUAL LAYOUT DESCRIPTION

Top-down. The agent (orange) has two arrows in: a **write** path (left) flowing through the embedder into Postgres, and a **read** path (right) flowing through hybrid search and the decay scorer back out. The two retrieval boxes are grouped to show recall = **relevance** × **freshness**. Single green datastore underscores 'no separate vector DB needed'.

MNEMO · SECTION 09

Interview Yield ★★★★★

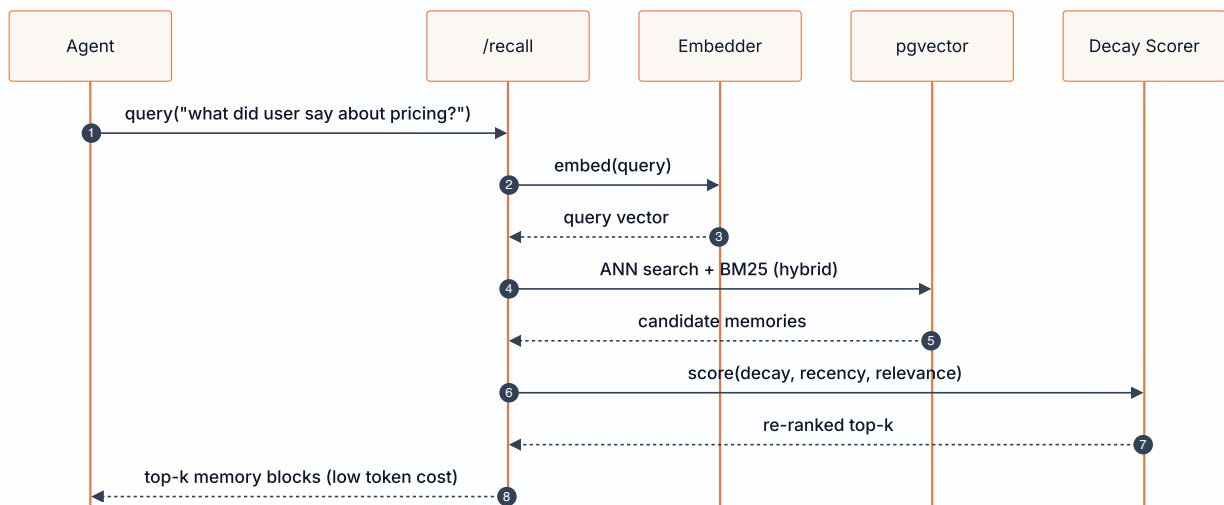
Request Lifecycle

Diagram 2 — End-to-End Request Flow

09

● Recall — Request Lifecycle

SEQUENCE



○ ERASER DIAGRAM SPEC

```
Agent > recall: query
recall > Embedder: embed
Embedder > recall: vector
recall > pgvector: ANN + BM25
pgvector > recall: candidates
recall > Decay: score
Decay > recall: top-k
recall > Agent: memory blocks
```

■ COMPONENT BREAKDOWN

- Embed query** Same model as write — vectors live in one space.
- Hybrid retrieve** ANN catches semantics; BM25 catches exact terms/IDs.
- Decay re-rank** Old, unused memories sink; fresh, relevant ones rise.
- Top-k only** Returns a handful of blocks, not the whole history.

▣ VISUAL LAYOUT DESCRIPTION

Five-actor sequence with a clean left-to-right pipeline feel. The **hybrid search** message to pgvector is the widest hop; the **decay scoring** step is a distinct stage before return. The final dashed arrow back to the Agent is annotated 'low token cost' — the entire value proposition in one label.

MNEMO · SECTION 10

Interview Yield ★★★★★

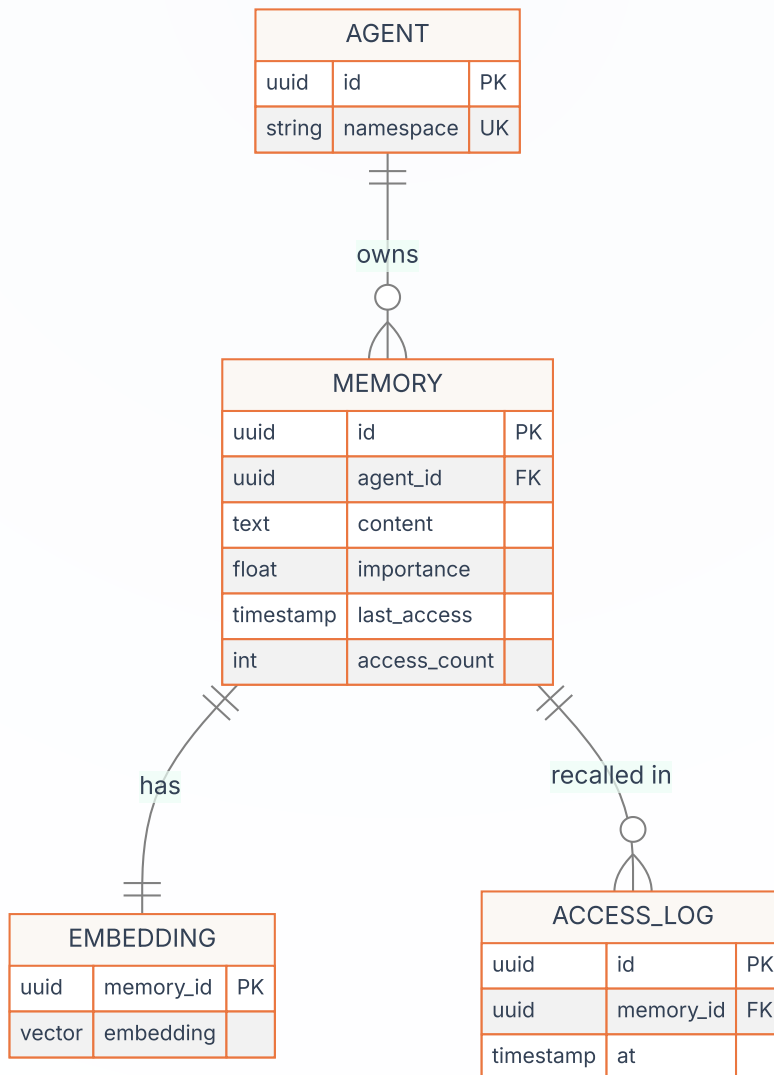
Database Design

Diagram 3 — Entity Relationship Model

10

● Mnemo — Data Model

ERD



○ ERASER DIAGRAM SPEC

```
AGENT { id pk; namespace uk }
MEMORY { id pk; agent_id fk; content; importance;
last_access; access_count }
EMBEDDING { memory_id pk; embedding vector }
ACCESS_LOG { id pk; memory_id fk; at }
AGENT 1-* MEMORY
MEMORY 1-1 EMBEDDING
MEMORY 1-* ACCESS_LOG
```

■ COMPONENT BREAKDOWN

- vector column** pgvector type with an IVFFlat/HNSW index for ANN.
- Decay inputs** last_access + access_count + importance feed the forgetting curve.
- 1:1 embedding** Separated so the wide vector doesn't bloat row scans.
- Access log** Each recall updates recency — reinforcing useful memories.

■ VISUAL LAYOUT DESCRIPTION

AGENT top, fanning to **MEMORY** (center). **MEMORY** links 1:1 down to **EMBEDDING** (the vector, set apart deliberately) and 1:N to **ACCESS_LOG**. The decay-driving columns on **MEMORY** (last_access, access_count, importance) are highlighted — the schema visibly encodes the Ebbinghaus inputs.

MNEMO · SECTION 11

Interview Yield ★★★★★

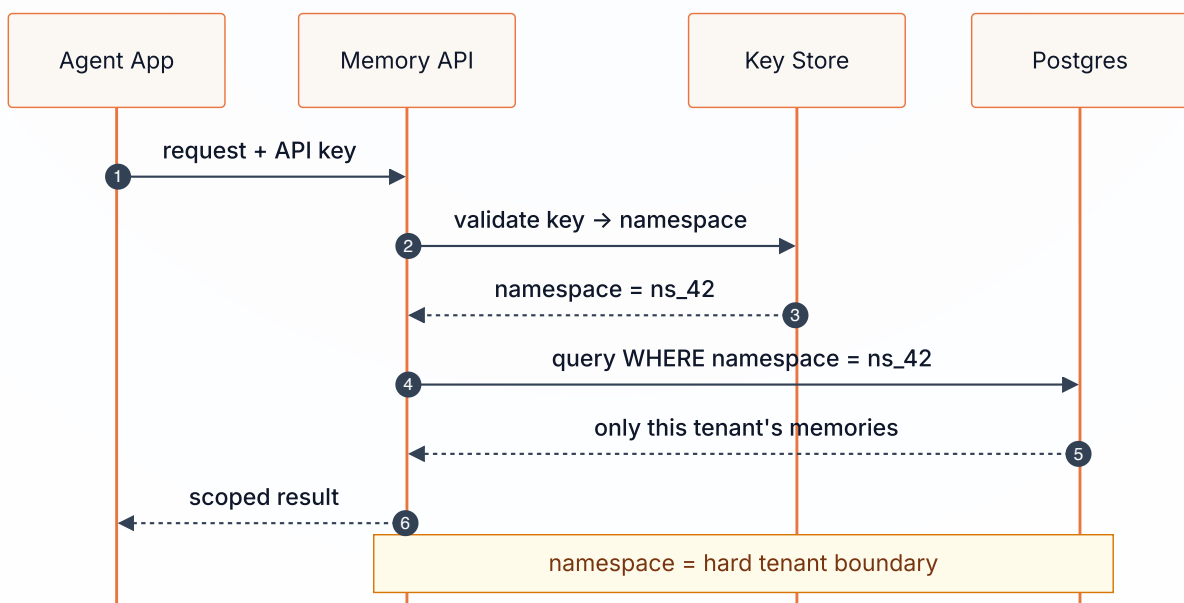
Authentication Design

Diagram 4 — Identity & Authorization Flow

11

● API-Key + Namespace Isolation

SEQUENCE



○ ERASER DIAGRAM SPEC

Agent > API: API key
API > KeyStore: validate
KeyStore > API: namespace
API > Postgres: WHERE namespace=ns
Postgres > API: scoped memories
API > Agent: result

▣ COMPONENT BREAKDOWN

API key Per-tenant secret, hashed at rest.
Namespace map Key resolves to exactly one namespace.
Hard filter Every query is namespace-scoped — no cross-tenant leakage.
Rate tie-in Keys also carry per-tenant rate limits.

▣ VISUAL LAYOUT DESCRIPTION

Four lifelines. The **validate key → namespace** hop is the trust gate; everything after it is scoped by namespace. A spanning note under API/Postgres states 'namespace = hard tenant boundary' — memories are sensitive, so isolation is the headline.

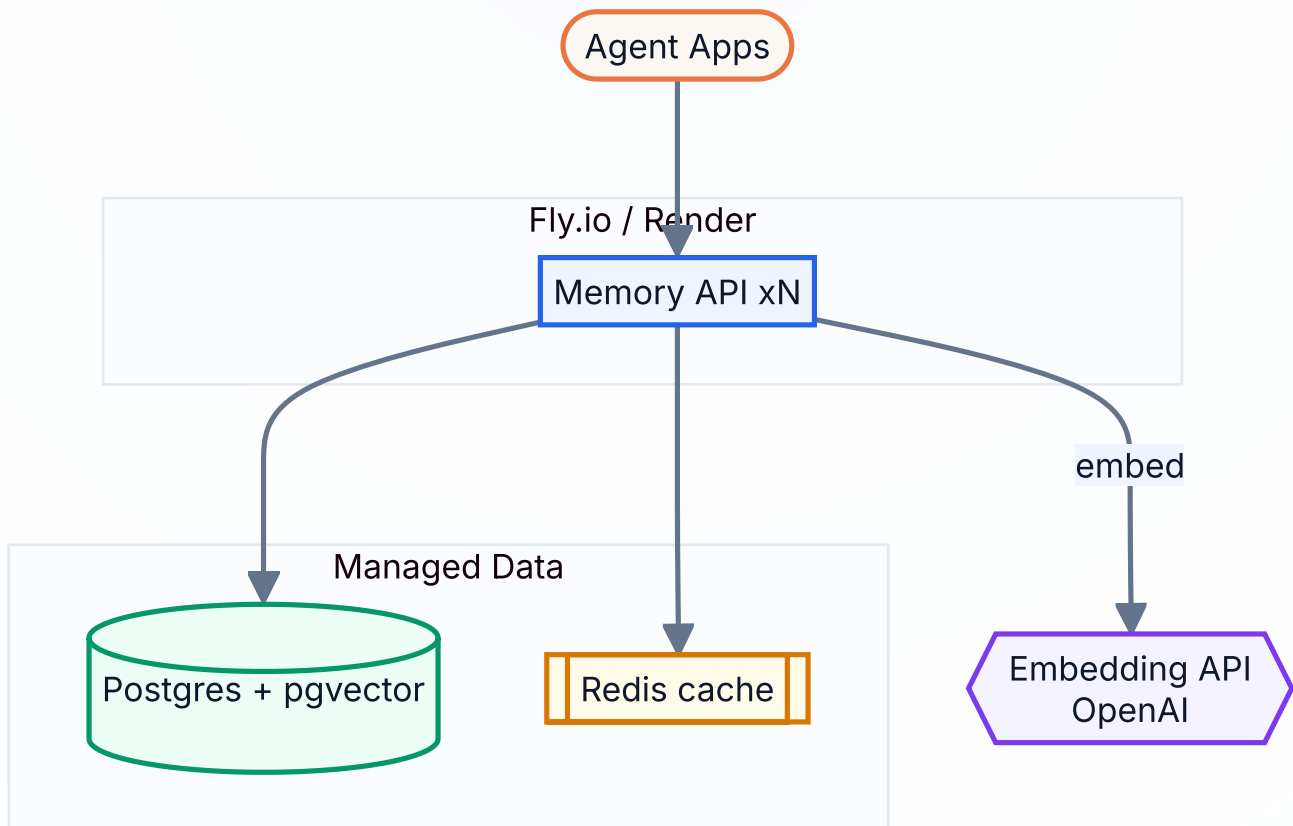
Deployment Architecture

Diagram 5 — Cloud Deployment Topology

12

● Cloud Deployment Topology

DEPLOYMENT



Client / Edge Service Data store Async / Queue External API

○ ERASER DIAGRAM SPEC

Agents > Memory API xN
API > Redis cache
API > Postgres+pgvector
API > Embedding API
// zones: Fly.io | Managed Data

▣ COMPONENT BREAKDOWN

API replicas Stateless; scale with request volume.
Redis cache Caches hot query embeddings + frequent recalls.
pgvector ANN index lives with the data — one system to operate.
Embedding API External; cached aggressively to cut spend.

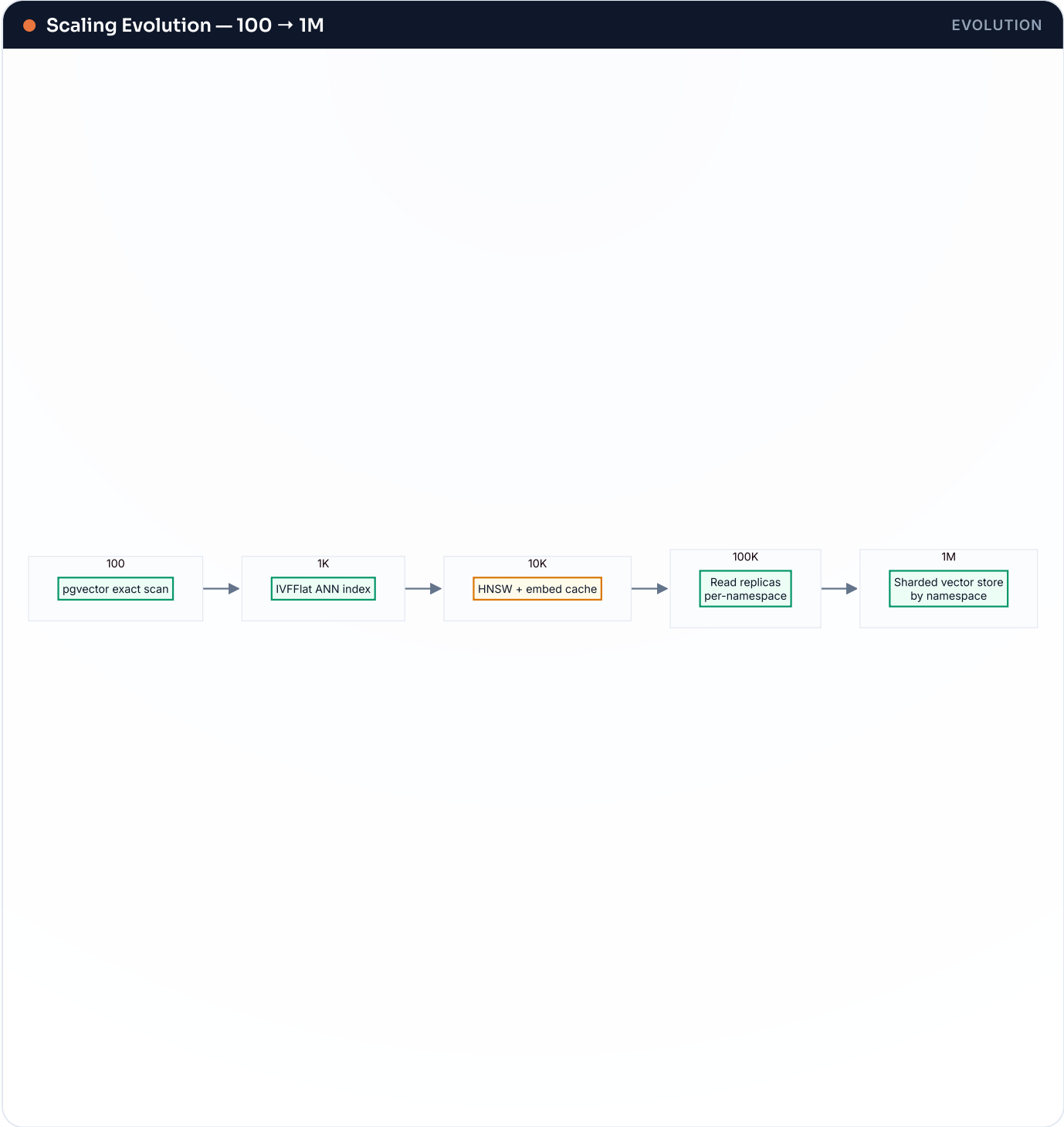
▣ VISUAL LAYOUT DESCRIPTION

Compact two-zone deployment. Stateless **API xN** on the left platform; **managed data** (pgvector + Redis) center; the external **embedding API** floats right as a hexagon. The Redis cache between API and the embedding API signals the cost strategy — don't re-embed what you've seen.

Scalability Architecture

Diagram 6 — Scaling Evolution 100 → 1M

13



Scale	First Bottleneck	Architectural Move
100	None — trivial	Exact cosine scan
1K	Linear scan latency	IVFFlat ANN index
10K	Recall speed	HNSW index + embed cache
100K	Read load	Per-namespace read replicas
1M	Single-index limits	Shard vector store by namespace

Security Architecture

Threat model & controls

14

Memories often contain personal data, so isolation and expiry are security-critical: a namespace is a hard wall, and sensitive memories can carry a TTL.

Threat (STRIDE)	Vector	Mitigation
Info Disclosure	Cross-tenant memory recall	Namespace filter on every query
Tampering	Forged API key	Hashed keys + per-key namespace binding
Retention	PII kept indefinitely	TTL hard-expiry + delete API
DoS	Recall flooding	Per-key rate limits



Memory is sensitive by default — Namespaces are hard boundaries and sensitive memories get a TTL — recall can never cross tenants or outlive its retention window.

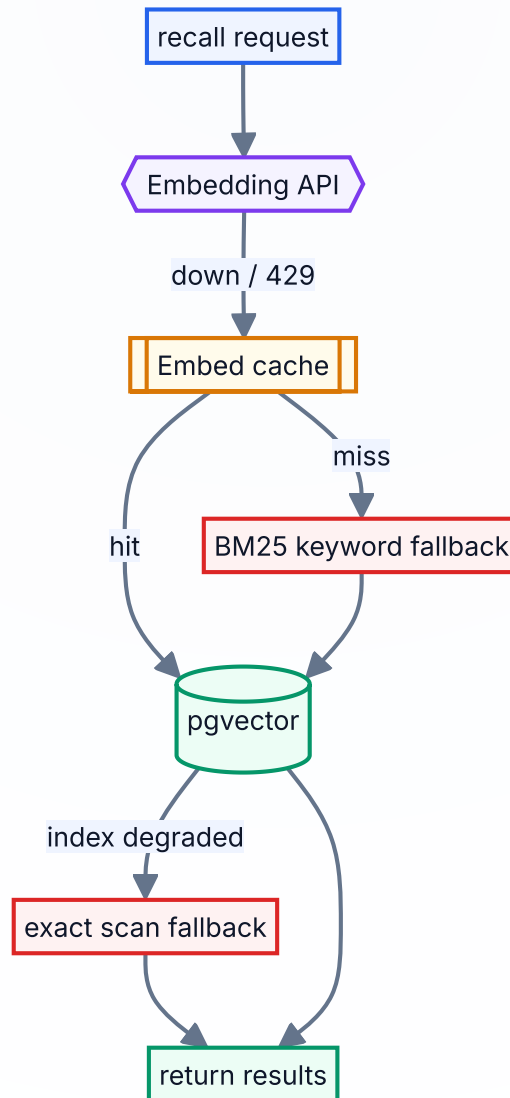
Failure Recovery

Diagram 7 — Failure & Recovery Topology

15

Failure & Recovery Topology

RESILIENCE



Healthy Failed / Degraded Recovery path

ERASER DIAGRAM SPEC

```
recall > Embedding API
Embedding down > cache
cache hit > pgvector
cache miss > BM25 fallback > pgvector
pgvector index degraded > exact scan
> return results
```

COMPONENT BREAKDOWN

- Embed-API outage** Cache serves vectors; if miss, fall back to BM25 keyword search.
- Graceful recall** Keyword results beat no results — degrade, don't fail.
- Index degrade** If ANN index is rebuilding, exact scan still answers (slower).
- Always answers** Every failure path still returns memories.

VISUAL LAYOUT DESCRIPTION

A failure decision tree where **every leaf returns results** — the resilience thesis is 'never return empty'. Embedding outage routes to cache then BM25 (red fallback) but still reaches green pgvector. Index degradation routes to exact scan. Red = degraded-but-working, never dead.

Monitoring & Observability

Golden signals + alert matrix

16

Instrumented around the four golden signals. Every alert is actionable and maps to a runbook.

**Latency**

Recall p99, embed time.

**Traffic**

Recalls/min, writes/min.

**Errors**Embed-API failures,
cache misses.**Saturation**

Index size, PG CPU.

ALERTING MATRIX

Signal	Threshold	Action / Runbook
Embed-API errors > 2%	1m	Serve from cache; switch provider
Recall p99 > 300ms	5m	Rebuild/retune ANN index
Cache hit < 60%	sustained	Grow cache; review key entropy
Index bloat	weekly	Schedule VACUUM / reindex

Cost Optimization

Unit economics under load

17

Two cost centers: **embedding calls** and **prompt tokens**. Mnemo attacks both — cache embeddings so we don't pay twice, and return top-k so prompts stay small.

-70%

PROMPT TOKENS

Cache

NO RE-EMBEDDING

1 DB

NO VECTOR-DB BILL

LEVERS PULLED

- ✓ Cache query + content embeddings in Redis
- ✓ Return top-k blocks, never full history
- ✓ pgvector instead of a paid vector DB
- ✓ Batch embedding writes to amortize API calls

Engineering Challenges

What broke, why, and what we learned

18

PRODUCTION INCIDENT — POST-MORTEM



The Incident

The Stale-Memory Bug. Agents kept resurfacing a months-old, contradicted fact ('user prefers email') over a newer one ('user prefers Slack'), because pure cosine similarity ranked the older memory higher.



Root Cause

Ranking was similarity-only. It had no notion of **time or reinforcement**, so an old but semantically-close memory could permanently outrank a fresh, correct one.



The Fix

Added the **Ebbinghaus decay scorer**: final score = $\text{relevance} \times e^{(-\Delta t/S)} \times \text{importance}$, where S grows with access_count. Reinforced, recent memories win.



Lasting Lesson

Relevance is not similarity. Good recall blends semantic match with recency and how often a memory has proven useful.



Blast radius — Agents gave confidently outdated answers across all tenants until decay shipped. The fix changed ranking globally with no schema migration.

Every architecture is a set of deliberate sacrifices. Here are the ones we made on purpose — and what we gave up.

Decision	We Chose	We Sacrificed
Store	pgvector (one DB)	Specialized-vector-DB speed at huge scale
Recall	Hybrid + decay	Pure-vector simplicity — but worse relevance
Context	Top-k blocks	Full recall — but 70% more tokens
Embeddings	External API	Self-hosted control — but more ops

WHERE WE OPTIMIZED



Future Roadmap

Where the architecture goes next

20

The roadmap deepens the cognitive model — memories that summarize, merge, and forget like a real memory system.

- V2 Memory Consolidation**
Periodically merge related memories into higher-level summaries to compress the store.
- V3 Self-Hosted Embeddings**
Optional local embedding model to remove the external API dependency and cost.
- V4 Graph Memory**
Link memories into an entity graph for multi-hop recall (who → relates-to → what).

Interview Deep Dive

The questions that decide the loop

21

Q Why store vectors in Postgres instead of a dedicated vector DB?

Operational simplicity. pgvector keeps embeddings, relational metadata, and keyword search in one system — one backup, one security model, one transaction. A separate vector DB adds a sync problem and a second thing to operate. We'd only split it out past ~1M vectors per namespace where specialized indexes win.

[pgvector](#)[Operational cost](#)[ANN](#)

Q How does the Ebbinghaus decay scoring work?

Final rank = relevance $\times e^{(-\Delta t/S)}$ $\times$ importance. Δt is time since last access; S (memory strength) grows with access_count. So a memory that's used often decays slowly and stays recallable, while an untouched one fades — mirroring the human forgetting curve.

[Decay](#)[Forgetting curve](#)[Re-ranking](#)

Q Why hybrid search instead of pure vector similarity?

Vectors capture **semantics** but miss exact tokens — IDs, names, error codes. BM25 keyword search nails those. Running both and fusing the scores gives recall that's both meaning-aware and precise on literals.

[Hybrid search](#)[BM25](#)[Vector ANN](#)

Q How do you keep recall cheap as history grows?

We never inject full history — only the **top-k** blocks after ranking. Memory cost is bounded by k , not conversation length, so token spend stays roughly flat even as the store grows to millions of memories.

[Top-k](#)[Token budget](#)[Sublinear cost](#)

Revision Sheet

One-page recall before the round

22

Mental model: **memory = relevance × freshness**, stored in pgvector, returned top-k. Cheap, sharp context.

ARCHITECTURE

Agent → /remember (embed) → pgvector; /recall → hybrid search → decay scorer → top-k blocks.

RANKING

Ebbinghaus: $\text{relevance} \times e^{(-\Delta t/S)} \times \text{importance}$; S grows with access_count.

DATABASE

AGENT → MEMORY → EMBEDDING(1:1, vector col) → ACCESS_LOG; decay inputs on MEMORY.

SCALING

exact → IVFFlat → HNSW → replicas → shard-by-namespace. A vector-index story.

COST

Cache embeddings + return top-k → ~70% fewer prompt tokens; pgvector avoids a vector-DB bill.

SIGNATURE BUG

Stale memory outranked fresh → fixed with decay-weighted re-ranking.



60-second pitch — Mnemo is long-term memory for LLM agents. It embeds facts into pgvector, recalls them with hybrid vector+keyword search, then re-ranks by an Ebbinghaus forgetting curve so fresh, reinforced memories win. Agents inject only top-k blocks, cutting prompt token cost ~70% while staying coherent across sessions — all on a single Postgres.

04

PROJECT DOSSIER · 04

ROVN

AI-Powered Lead Ingestion workspace with Intent Scoring & Prioritization.

SalesTech	Kafka + LLM	40%	Tier 2
DOMAIN	CORE	CONV. LIFT	PRIORITY

ROVN · SECTION 01

Interview Yield ★★★★★

Executive Summary

What it is, who it serves, why it matters

01

ROVN uses AI to score and prioritize inbound leads.

40%

CONV. LIFT

**The Core Problem**

Too much noise in inbound leads.

**Target Users & Value**

SDRs.

**Business Goal & Impact**

Prioritize hot leads.

**My Role & Ownership**

Architect.

Product Vision

The bet, the wedge, the moat

02

Dynamic AI scoring.

ALTERNATIVES WE REJECTED

Alternative	Where It Breaks	ROVN Edge
Manual	Slow.	Fast.

CORE CAPABILITY MATRIX



Score
Real time.

Business Problem

Quantifying the pain we remove

03

Time kills deals.



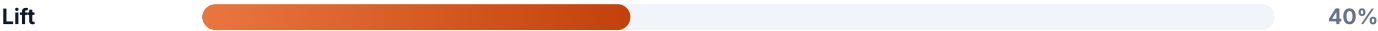
Spam

Too much.



Why now — Cheap LLMs.

COST OF THE STATUS QUO



User Journey

From first touch to recurring value

04

Lead comes in, gets scored, SDR calls.

- 1

Ingest

Webhook.

Functional Requirements

P0 / P1 capability contract

05

Scoped to the critical path. Each requirement maps to an owned subsystem and an acceptance signal.

MUST-HAVE (P0)

SHOULD-HAVE (P1)

- ✓

Ingestion
- ›

Reporting

ID	Capability	Acceptance Signal
FR1	Ingest	Webhook

Non-Functional Requirements

SLOs, not vibes

06

The quality attributes that define production-readiness — each with a measurable target and the mechanism that enforces it.

Attribute	Target (SLO)	Enforcement Mechanism
Perf	Fast	Kafka

QUALITY ATTRIBUTE PRIORITY



Tech Stack Decisions

Every choice has a bill

07

Kafka + LLM.

Layer	Chosen	Why This	Trade-off Accepted
Stream	Kafka	Durable	Setup

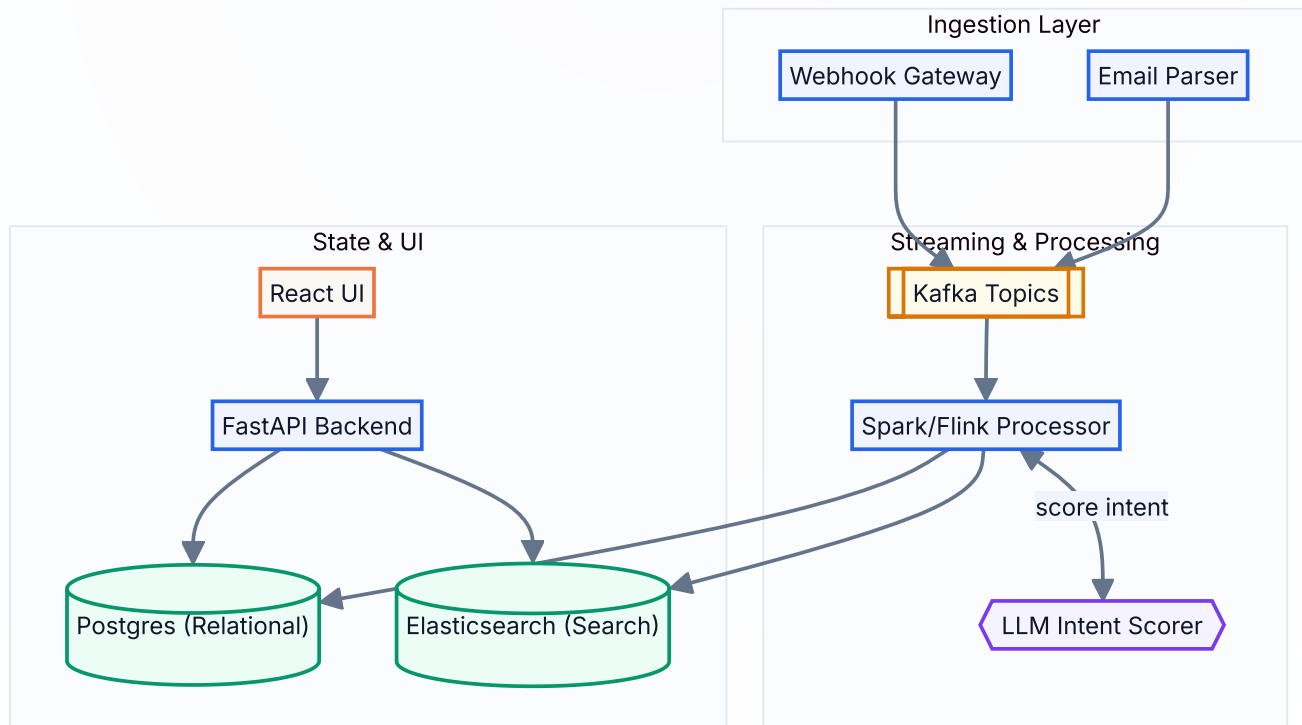
Architecture Overview

Diagram 1 — System Architecture

08

● ROVN — System Architecture

C4 / CONTAINER



Client / Edge Service Data store Async / Queue External API

○ ERASER DIAGRAM SPEC

Webhook [icon: webhook]
Email [icon: mail]
Kafka [icon: kafka, color: amber]
Processor [icon: cpu, color: blue]
LLM [icon: openai, color: purple]
Postgres [icon: postgresql, color: green]
Elasticsearch [icon: elastic, color: green]
API [icon: python, color: blue]
UI [icon: react, color: orange]
Webhook > Kafka
Email > Kafka
Kafka > Processor
Processor <> LLM: score intent
Processor > Postgres
Processor > Elasticsearch
API > Postgres
API > Elasticsearch
UI > API

■ COMPONENT BREAKDOWN

Webhook/Email High-throughput ingestion endpoints.
Kafka Buffers bursts of incoming leads securely.
Processor Asynchronously evaluates intent using LLM.
Elasticsearch Powers fast text search on parsed leads.

▣ VISUAL LAYOUT DESCRIPTION

Top-to-bottom data flow from ingestion to storage, with UI pulling from state.

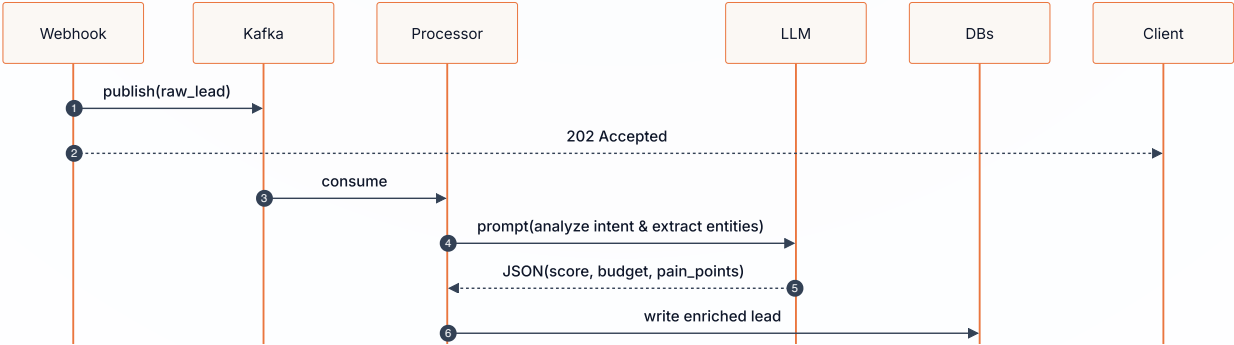
Request Lifecycle

Diagram 2 — End-to-End Request Flow

09

● Lead Ingestion Flow

SEQUENCE



○ ERASER DIAGRAM SPEC

Webhook > Kafka: publish
Webhook > Client: 202
Kafka > Processor: consume
Processor > LLM: prompt
LLM > Processor: JSON
Processor > DB: write

■ COMPONENT BREAKDOWN

202 Accepted Immediate ack, async processing.
LLM Extraction Converts unstructured text to structured JSON.

■ VISUAL LAYOUT DESCRIPTION

Linear pipeline from ingestion to DB.

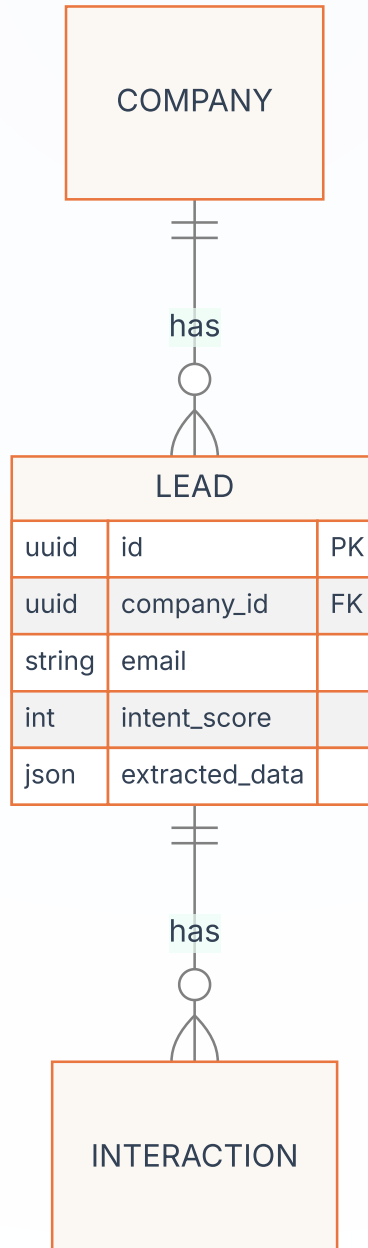
Database Design

Diagram 3 — Entity Relationship Model

10

● ROVN — Data Model

ERD



○ ERASER DIAGRAM SPEC

```
COMPANY { id pk }
LEAD { id pk; company_id fk; intent_score }
INTERACTION { id pk; lead_id fk }
COMPANY 1-* LEAD
LEAD 1-* INTERACTION
```

■ COMPONENT BREAKDOWN

intent_score Indexed for fast sorting by SDRs.

extracted_data JSONB column for flexible schema.

■ VISUAL LAYOUT DESCRIPTION

Standard hierarchy.

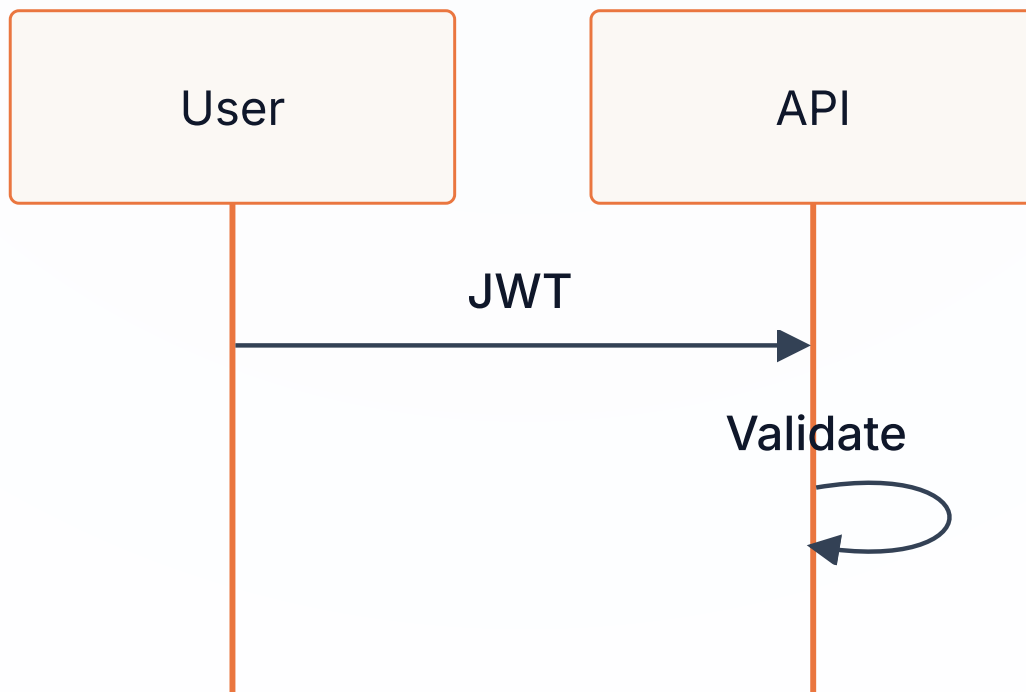
Authentication Design

Diagram 4 — Identity & Authorization Flow

11

● Auth

SEQUENCE



ERASER DIAGRAM SPEC

User > API: JWT

COMPONENT BREAKDOWN

JWT

Standard auth

VISUAL LAYOUT DESCRIPTION

Simple validation.

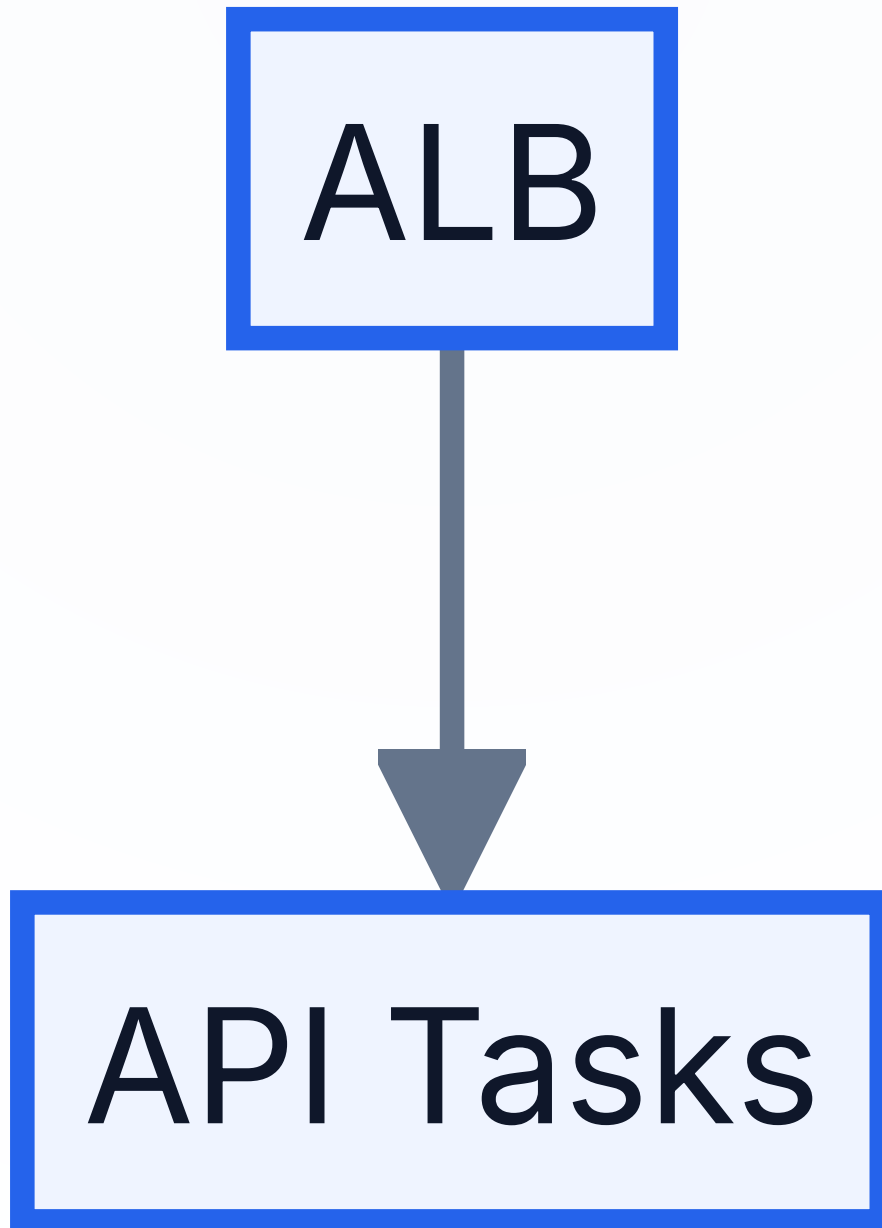
Deployment Architecture

Diagram 5 — Cloud Deployment Topology

12

● Deployment

DEPLOYMENT



Client / Edge Service Data store Async / Queue External API

○ ERASER DIAGRAM SPEC

ALB > API

▣ COMPONENT BREAKDOWN

ALB Load balancer

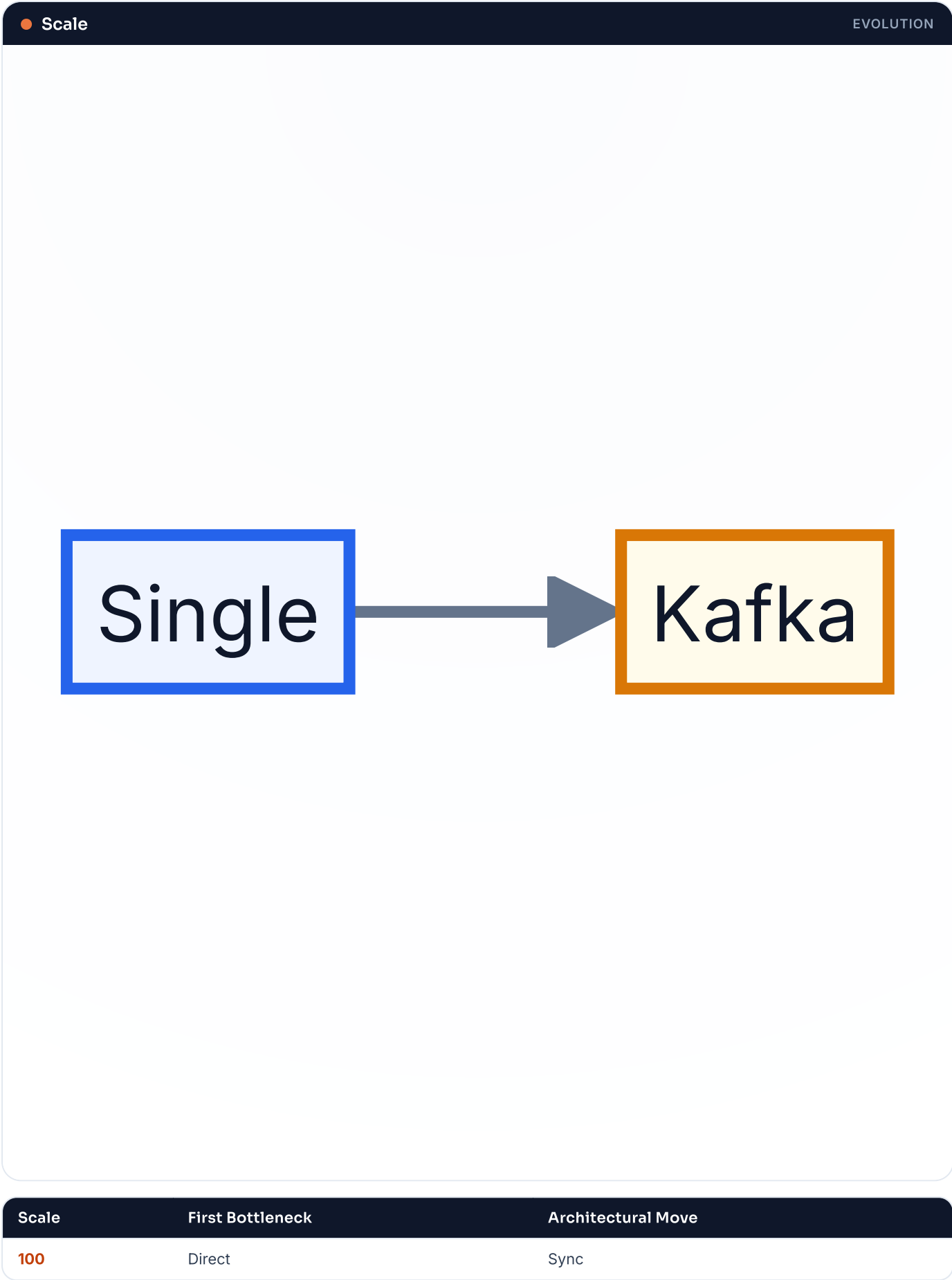
▣ VISUAL LAYOUT DESCRIPTION

Basic deploy.

Scalability Architecture

Diagram 6 — Scaling Evolution 100 → 1M

13



Security Architecture

Threat model & controls

14

Secure webhooks.

Threat (STRIDE)	Vector	Mitigation
Spam	Rate limit	WAF



Validate webhooks — Always validate webhooks to prevent spoofed data injection.

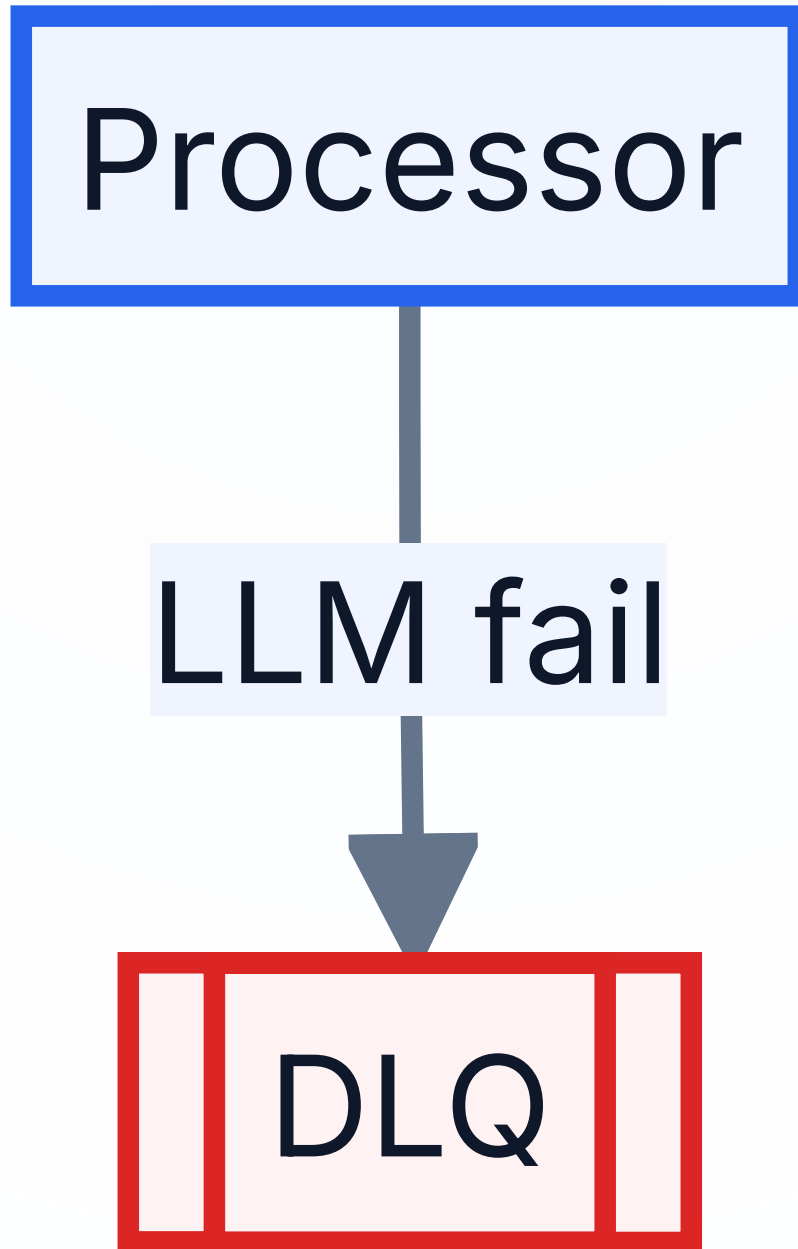
Failure Recovery

Diagram 7 — Failure & Recovery Topology

15

● Failure

RESILIENCE



● Healthy ● Failed / Degraded ● Recovery path

○ ERASER DIAGRAM SPEC

Processor > DLQ

▣ COMPONENT BREAKDOWN

DLQ

Handles LLM timeouts

▣ VISUAL LAYOUT DESCRIPTION

Failure flow.

Monitoring & Observability

Golden signals + alert matrix

16

Instrumented around the four golden signals. Every alert is actionable and maps to a runbook.



Latency
E2E

ALERTING MATRIX

Signal	Threshold	Action / Runbook
DLQ > 0	Check LLM	

ROVN · SECTION 17

Interview Yield ★★☆☆

Cost Optimization

Unit economics under load

17

Batch LLM calls.

Cheap

LLM

LEVERS PULLED

- ✓ Batch

Engineering Challenges

What broke, why, and what we learned

18

PRODUCTION INCIDENT — POST-MORTEM



The Incident

Kafka lag



Root Cause

The broker failed



The Fix

Add partition



Lasting Lesson

Scale out brokers



Blast radius — Delayed leads.

Architecture Tradeoffs

Chosen sacrifices, stated plainly

19

Every architecture is a set of deliberate sacrifices. Here are the ones we made on purpose — and what we gave up.

Decision	We Chose	We Sacrificed
Async	Consistency	Lag

WHERE WE OPTIMIZED



ROVN · SECTION 20

Future Roadmap

Where the architecture goes next

Interview Yield ★★☆☆☆

20

Auto replies.

V2 Auto
Drafts.

Interview Deep Dive

The questions that decide the loop

21

Q Why Kafka?

Bursts.

Kafka

ROVN · SECTION 22

Interview Yield ★★★★★

Revision Sheet

One-page recall before the round

22

Kafka + LLM.

ARCH

Webhook → Kafka → LLM

**60-second pitch** — ROVN prioritizes leads using AI.

05

PROJECT DOSSIER · 05

Travio

Collaborative trip planner with Operational Transformation conflict sorting.

TravelTech

DOMAIN

WebSockets+OT

CORE

Realtime

SYNC

Tier 2

PRIORITY

Executive Summary

What it is, who it serves, why it matters

01

Travio is a real-time collaborative trip planner.

Sync

REALTIME



The Core Problem

Conflicts in shared planning.



Target Users & Value

Travelers.



Business Goal & Impact

Seamless collab.



My Role & Ownership

Backend.

Product Vision

The bet, the wedge, the moat

02

OT-based sync.

ALTERNATIVES WE REJECTED

Alternative	Where It Breaks	Travio Edge
Google Docs	Generic.	Specific.

CORE CAPABILITY MATRIX



Sync
Live.

Business Problem

Quantifying the pain we remove

03

Collab.



Conflict

Overwrites.



Why now — Yjs.

COST OF THE STATUS QUO



User Journey

From first touch to recurring value

04

Plan together.

1 Plan
Edit.

Functional Requirements

P0 / P1 capability contract

05

Scoped to the critical path. Each requirement maps to an owned subsystem and an acceptance signal.

- MUST-HAVE (P0)

✓ Sync
- SHOULD-HAVE (P1)

› Export

ID	Capability	Acceptance Signal
FR1	Sync	Live

Non-Functional Requirements

SLOs, not vibes

06

The quality attributes that define production-readiness — each with a measurable target and the mechanism that enforces it.

Attribute	Target (SLO)	Enforcement Mechanism
Perf	Low lat	WS

QUALITY ATTRIBUTE PRIORITY



Tech Stack Decisions

Every choice has a bill

07

WS + OT.

Layer	Chosen	Why This	Trade-off Accepted
RT	Yjs	OT	Complex

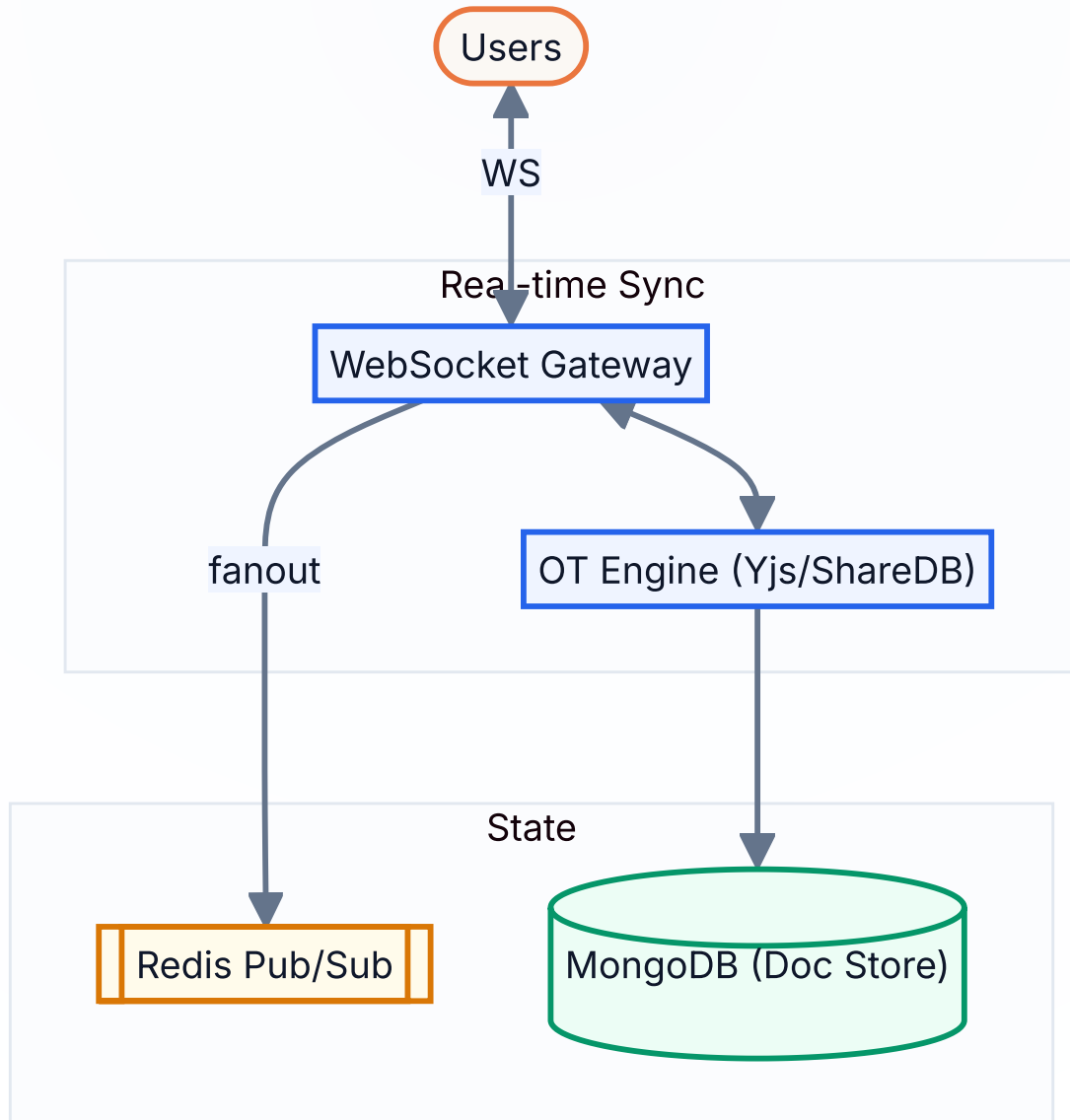
Architecture Overview

Diagram 1 — System Architecture

08

● Travio — System Architecture

C4 / CONTAINER



Client / Edge Service Data store Async / Queue External API

○ ERASER DIAGRAM SPEC

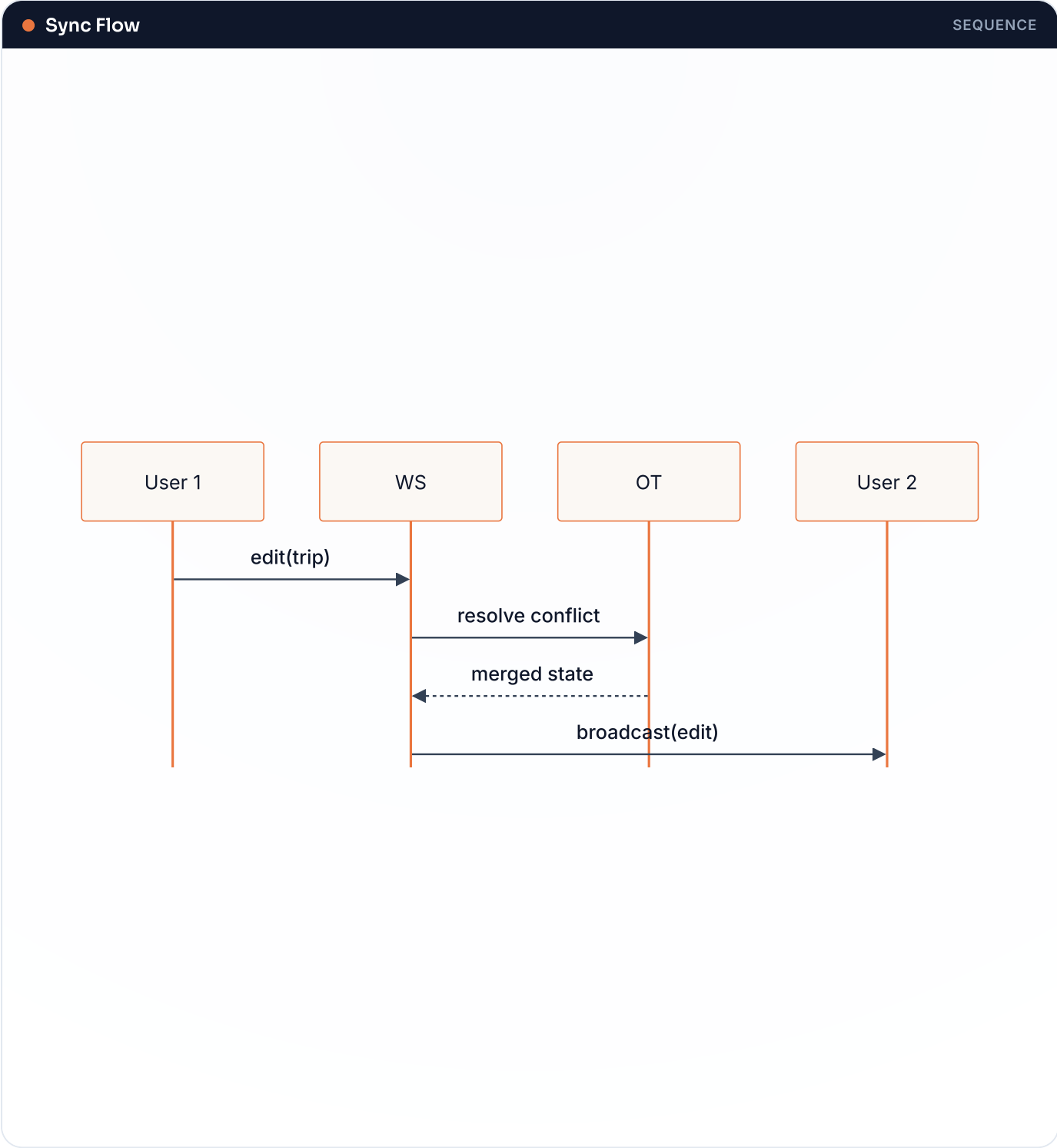
```
Users [icon: users]
WS [icon: socket, color: blue]
OT [icon: cpu, color: blue]
Redis [icon: redis, color: amber]
Mongo [icon: mongodb, color: green]
Users <-> WS
WS <-> OT
WS > Redis
OT > Mongo
```

■ COMPONENT BREAKDOWN

OT Engine	Handles concurrent document edits.
WebSockets	Bi-directional sync.
MongoDB	Stores JSON documents of trips.

▣ VISUAL LAYOUT DESCRIPTION

Real-time loop.



○ ERASER DIAGRAM SPEC

U1 > WS: edit
WS > OT: resolve
OT > WS: merged
WS > U2: broadcast

■ COMPONENT BREAKDOWN

OT Conflict resolution.

▣ VISUAL LAYOUT DESCRIPTION

Sync sequence.

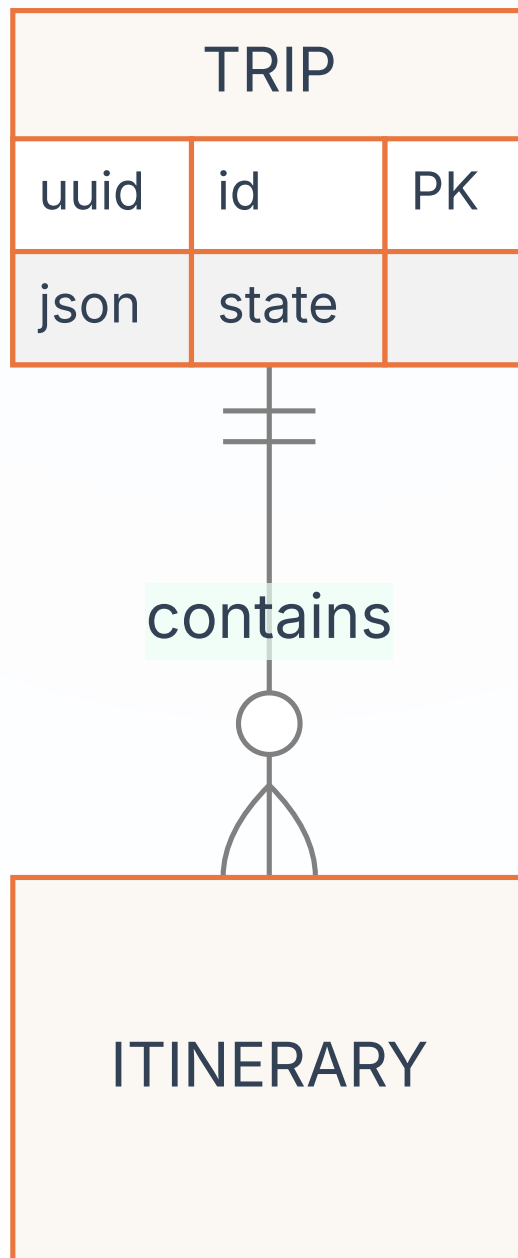
Database Design

Diagram 3 — Entity Relationship Model

10

● Travio — Data Model

ERD



○ ERASER DIAGRAM SPEC

TRIP { id pk; state json }

▣ COMPONENT BREAKDOWN

state

JSON doc.

▣ VISUAL LAYOUT DESCRIPTION

Simple doc.

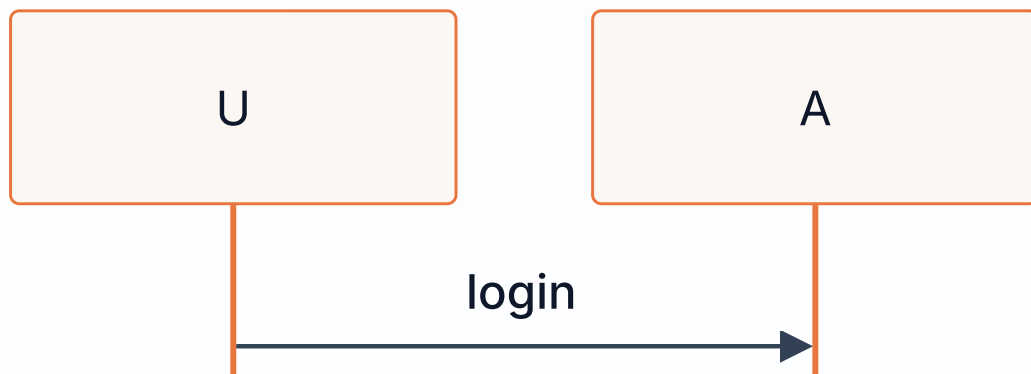
Authentication Design

Diagram 4 — Identity & Authorization Flow

11

● Auth

SEQUENCE



ERASER DIAGRAM SPEC

U > A

COMPONENT BREAKDOWN

Auth ...

VISUAL LAYOUT DESCRIPTION

...

Deployment Architecture

Diagram 5 — Cloud Deployment Topology

12

Deploy

DEPLOYMENT



Client / Edge

Service

Data store

Async / Queue

External API

ERASER DIAGRAM SPEC

W

COMPONENT BREAKDOWN

WS ...

VISUAL LAYOUT DESCRIPTION

...

Scalability Architecture

Diagram 6 — Scaling Evolution 100 → 1M

13

Scale

EVOLUTION



Scale	First Bottleneck	Architectural Move
100	Node	Single

Auth.

Threat (STRIDE)	Vector	Mitigation
Access	Token	Valid

 **Secure WS** — Authenticate web sockets during connection handshake.

Failure Recovery

Diagram 7 — Failure & Recovery Topology

15

● Failure

RESILIENCE



● Healthy
 ● Failed / Degraded
 ● Recovery path

○ ERASER DIAGRAM SPEC

Fail

▣ COMPONENT BREAKDOWN

Fail

...

▣ VISUAL LAYOUT DESCRIPTION

...

Monitoring & Observability

Golden signals + alert matrix

16

Instrumented around the four golden signals. Every alert is actionable and maps to a runbook.



Sync Lat
ms

ALERTING MATRIX

Signal	Threshold	Action / Runbook
WS Drop	Check	

TRAVIO · SECTION 17

Cost Optimization

Unit economics under load

Interview Yield ★★☆☆

17

WS conns.

Low

COST

LEVERS PULLED

- ✓ Scale

Engineering Challenges

What broke, why, and what we learned

18

PRODUCTION INCIDENT — POST-MORTEM



The Incident

OT split



Root Cause

Concurrency conflict



The Fix

CRDT merge



Lasting Lesson

Use CRDTs



Blast radius — Desync.

Every architecture is a set of deliberate sacrifices. Here are the ones we made on purpose — and what we gave up.

Decision	We Chose	We Sacrificed
OT	CRDT	OT



TRAVIO · SECTION 20

Future Roadmap

Where the architecture goes next

Interview Yield ★★☆☆☆

20

AI planner.

V2 AI
Suggest.

Interview Deep Dive

The questions that decide the loop

21

Q Why OT?

Collab.

OT

Live sync.

ARCH

WS → OT → DB

✓ 60-second pitch — Travio makes planning collaborative.

06

PROJECT DOSSIER • 06

GrindOS

Offline-first Electron learning desktop app with IndexedDB replication.

EdTech

DOMAIN

Offline-First

CORE

Local Sync

SYNC

Tier 3

PRIORITY

Executive Summary

What it is, who it serves, why it matters

01

GrindOS is an offline-first learning platform.

Oms

READ LATENCY



The Core Problem

Students drop off due to poor internet.



Target Users & Value

Students in low-bandwidth areas.



Business Goal & Impact

Uninterrupted learning.



My Role & Ownership

Full-Stack.

Product Vision

The bet, the wedge, the moat

02

Local-first architecture.

ALTERNATIVES WE REJECTED

Alternative	Where It Breaks	GrindOS Edge
Web Only	Requires network.	Offline first.

CORE CAPABILITY MATRIX



Offline

Always works.

Business Problem

Quantifying the pain we remove

03

Accessibility.



No Net

Lost work.



Why now — PWA/Electron maturity.

COST OF THE STATUS QUO

Offline



Yes

GRINDOS · SECTION 04

User Journey

From first touch to recurring value

Interview Yield ★★★★★

04

Learn anywhere.

1 **Download**
Cache.

Functional Requirements

P0 / P1 capability contract

05

Scoped to the critical path. Each requirement maps to an owned subsystem and an acceptance signal.

MUST-HAVE (P0) Offline**SHOULD-HAVE (P1)** Sync

ID	Capability	Acceptance Signal
FR1	Offline	Local DB

Non-Functional Requirements

SLOs, not vibes

06

The quality attributes that define production-readiness — each with a measurable target and the mechanism that enforces it.

Attribute	Target (SLO)	Enforcement Mechanism
Perf	Fast	IDB

QUALITY ATTRIBUTE PRIORITY



Tech Stack Decisions

Every choice has a bill

07

Electron + IDB.

Layer	Chosen	Why This	Trade-off Accepted
Local	IDB	Fast	Complex

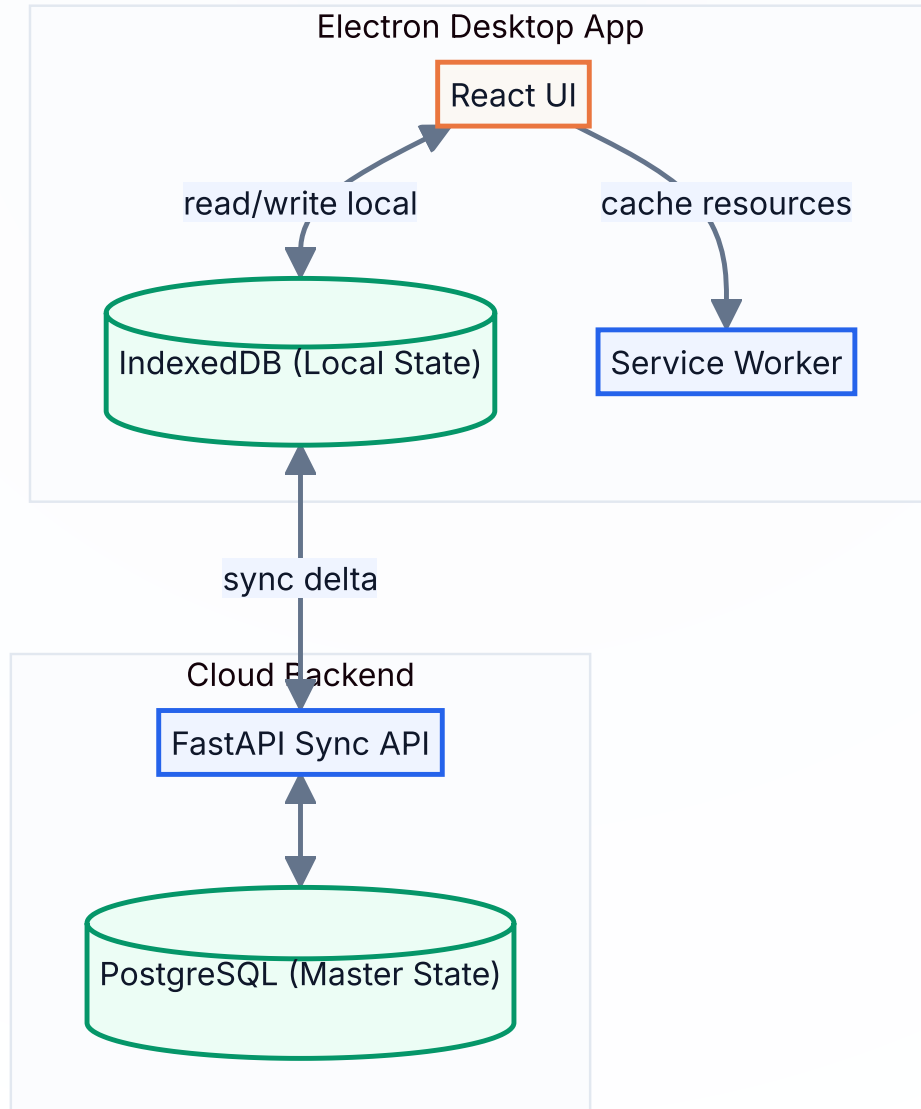
Architecture Overview

Diagram 1 — System Architecture

08

GrindOS Desktop — System Architecture

C4 / CONTAINER



Client / Edge Service Data store Async / Queue External API

ERASER DIAGRAM SPEC

UI [icon: react, color: orange]
IndexedDB [icon: database, color: green]
ServiceWorker [icon: cpu, color: blue]
Sync API [icon: python, color: blue]
Postgres [icon: postgresql, color: green]

UI <> IndexedDB: local read/write
UI > ServiceWorker: cache
IndexedDB <> Sync API: sync deltas
Sync API <> Postgres: master state

COMPONENT BREAKDOWN

IndexedDB Stores learning progress locally so the app works offline.

Service Worker Caches video/text assets for offline playback.

Sync API Merges local deltas with the master DB when online.

VISUAL LAYOUT DESCRIPTION

Left-to-right from local offline storage to cloud sync.

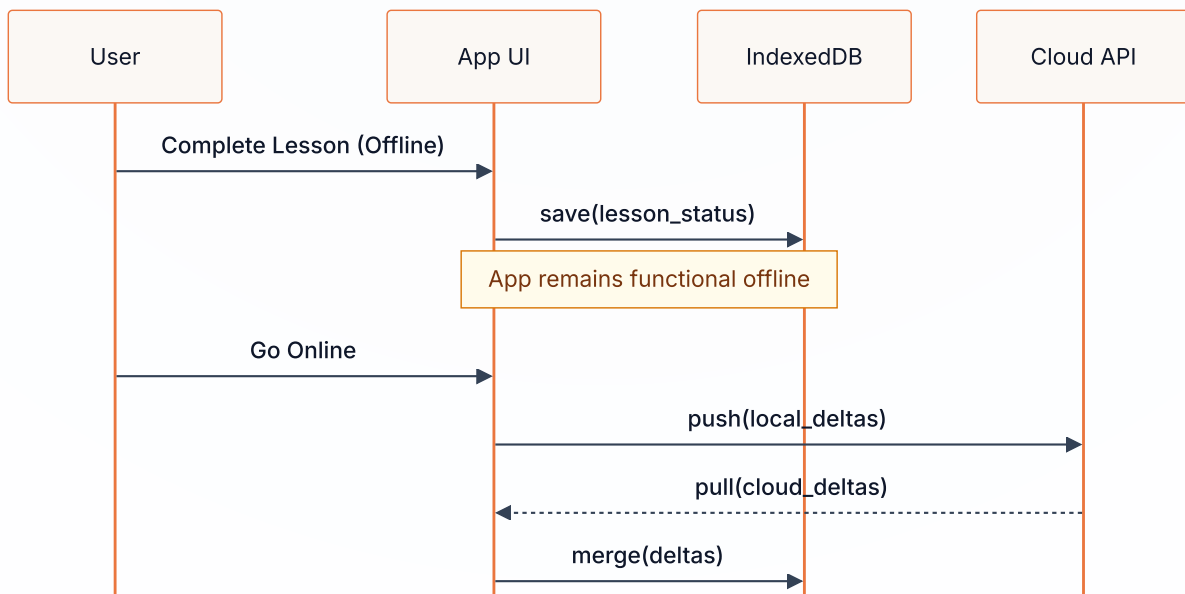
Request Lifecycle

Diagram 2 — End-to-End Request Flow

09

Offline Sync Flow

SEQUENCE



ERASER DIAGRAM SPEC

User > UI: complete lesson
UI > IndexedDB: save local
UI > API: go online → push deltas
API > UI: pull deltas
UI > IndexedDB: merge

COMPONENT BREAKDOWN

Local Write Zero-latency UI updates.
Delta Sync Transfers only changed rows to save bandwidth.

VISUAL LAYOUT DESCRIPTION

Offline actions buffered, then synced.

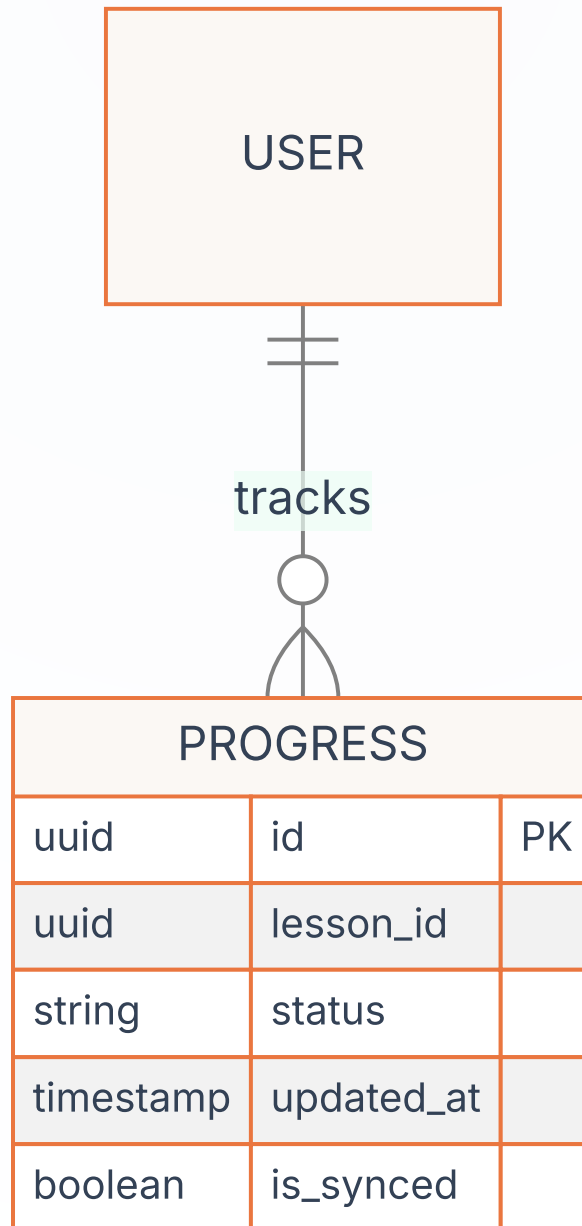
Database Design

Diagram 3 — Entity Relationship Model

10

● GrindOS — Data Model

ERD



○ ERASER DIAGRAM SPEC

```
USER { id pk }  
PROGRESS { id pk; lesson_id; status; updated_at;  
is_synced }  
USER 1-* PROGRESS
```

▣ COMPONENT BREAKDOWN

updated_at Used for conflict resolution (last-write-wins).
is_synced Flag to track pending uploads.

▣ VISUAL LAYOUT DESCRIPTION

Simple entity.

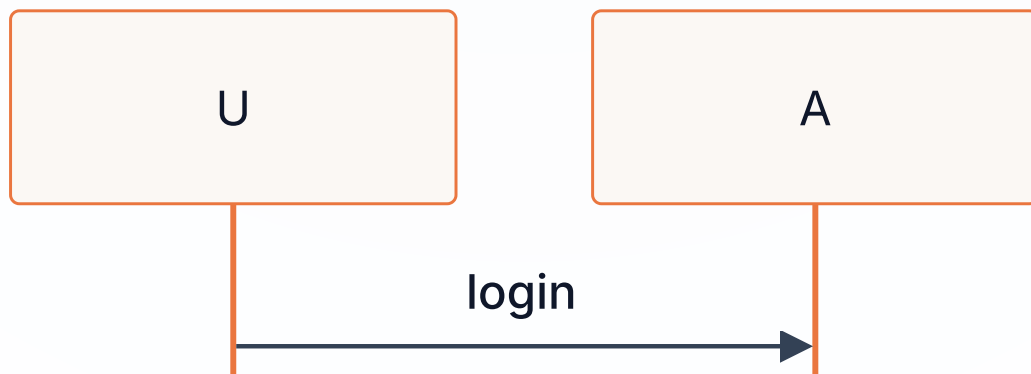
Authentication Design

Diagram 4 — Identity & Authorization Flow

11

● Auth

SEQUENCE



ERASER DIAGRAM SPEC

U > A

COMPONENT BREAKDOWN

JWT

Cached locally for offline access.

VISUAL LAYOUT DESCRIPTION

Standard

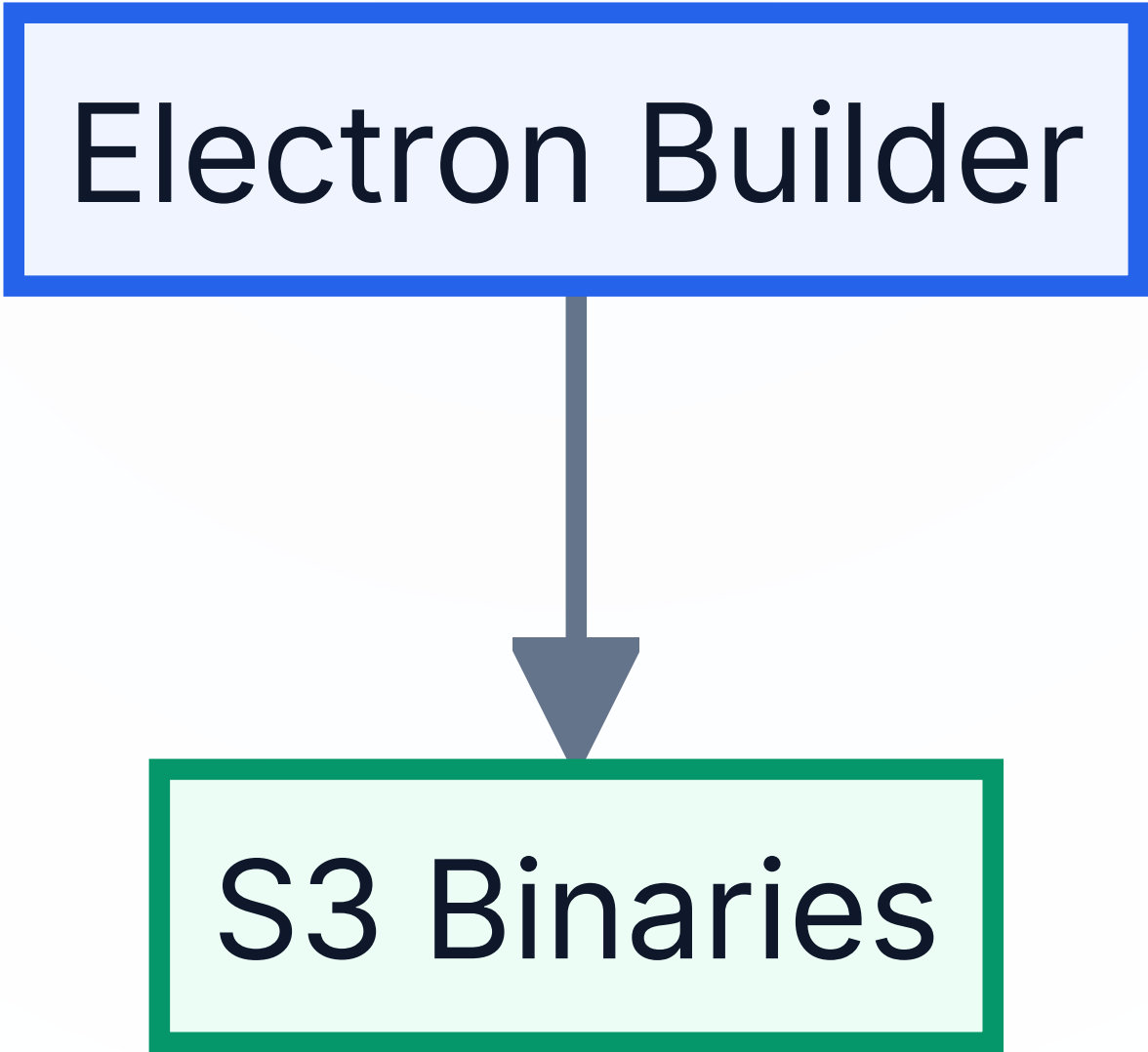
Deployment Architecture

Diagram 5 — Cloud Deployment Topology

12

Deploy

DEPLOYMENT



```
graph TD; EB[Electron Builder] --> S3[S3 Binaries];
```

Electron Builder

S3 Binaries

Client / Edge Service Data store Async / Queue External API

ERASER DIAGRAM SPEC

A > B

COMPONENT BREAKDOWN

Electron Builder Packages for Mac/Windows.

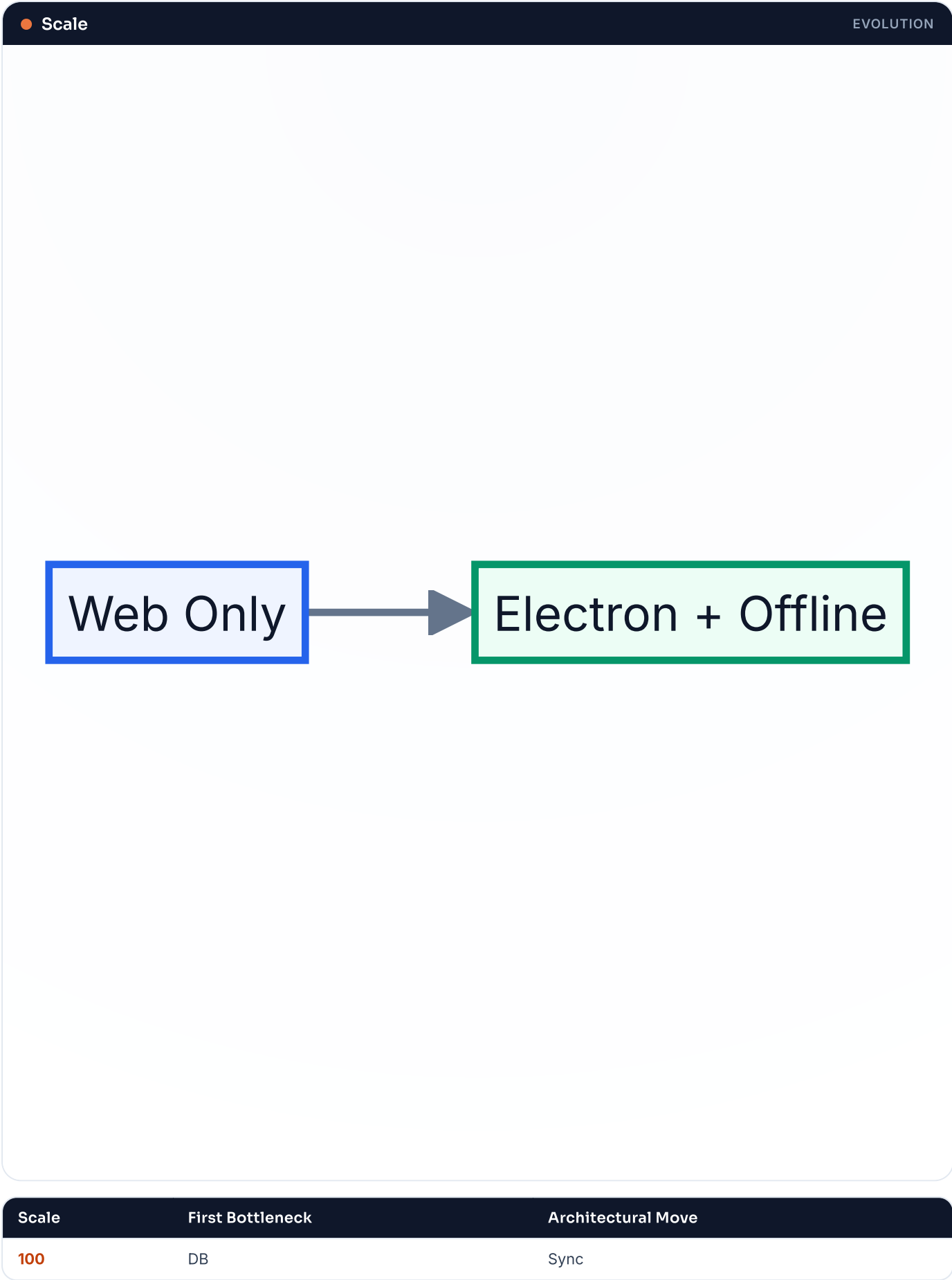
VISUAL LAYOUT DESCRIPTION

Simple CI/CD.

Scalability Architecture

Diagram 6 — Scaling Evolution 100 → 1M

13



Security Architecture

Threat model & controls

14

Local encryption.

Threat (STRIDE)	Vector	Mitigation
Local	Encrypt DB	AES



Secure local storage — Encrypt local IndexedDB using AES.

Failure Recovery

Diagram 7 — Failure & Recovery Topology

15

● Failure

RESILIENCE

Network Loss

Fallback

IndexedDB

● Healthy ● Failed / Degraded ● Recovery path

ERASER DIAGRAM SPEC

Net > IDB

COMPONENT BREAKDOWN

Graceful Degradation UI reads from local store.

VISUAL LAYOUT DESCRIPTION

Resilience.

Monitoring & Observability

Golden signals + alert matrix

16

Instrumented around the four golden signals. Every alert is actionable and maps to a runbook.



Sync Lat

ms

ALERTING MATRIX

Signal	Threshold	Action / Runbook
Sync Fail	Check API	

GRINDOS · SECTION 17

Interview Yield ★★☆☆

Cost Optimization

Unit economics under load

17

Client compute.

Low
COST

LEVERS PULLED

☒ Client Side

Engineering Challenges

What broke, why, and what we learned

18

PRODUCTION INCIDENT — POST-MORTEM



The Incident

Sync conflict



Root Cause

Offline divergence



The Fix

Last-write-wins



Lasting Lesson

Use LWW



Blast radius — Data loss.

Architecture Tradeoffs

Chosen sacrifices, stated plainly

19

Every architecture is a set of deliberate sacrifices. Here are the ones we made on purpose — and what we gave up.

Decision	We Chose	We Sacrificed
Local	State	Sync

WHERE WE OPTIMIZED

Local



Max

GRINDOS · SECTION 20

Future Roadmap

Where the architecture goes next

Interview Yield ★★☆☆☆

20

P2P sync.



P2P

WebRTC.

Interview Deep Dive

The questions that decide the loop

21

Q Why offline?

Access.

Offline

Revision Sheet

One-page recall before the round

22

Offline first.

ARCH

Electron → IDB → API



60-second pitch — GrindOS runs entirely offline, syncing only when you reconnect.

07

PROJECT DOSSIER · 07

Sankalan

Academic Resource Pathfinder with full-text search indexing.

EdTech

DOMAIN

PostgreSQL FTS

CORE

Fast Search

SYNC

Tier 3

PRIORITY

Executive Summary

What it is, who it serves, why it matters

01

Sankalan centralizes academic resources with instant search.

50ms

SEARCH LAT



The Core Problem

Scattered PDFs in WhatsApp groups.



Target Users & Value

University students.



Business Goal & Impact

Find resources instantly.



My Role & Ownership

Backend.

Product Vision

The bet, the wedge, the moat

02

Built-in Postgres FTS.

ALTERNATIVES WE REJECTED

Alternative	Where It Breaks	Sankalan Edge
Google Drive	No deep search.	FTS.

CORE CAPABILITY MATRIX



Search
Fast FTS.

Business Problem

Quantifying the pain we remove

03

Time saving.



Lost files

Chat history.



Why now — Postgres FTS maturity.

COST OF THE STATUS QUO



SANKALAN · SECTION 04

User Journey

From first touch to recurring value

Interview Yield ★★★★★

04

Type, click, download.

1 Search
Instant.

Functional Requirements

P0 / P1 capability contract

05

Scoped to the critical path. Each requirement maps to an owned subsystem and an acceptance signal.

- MUST-HAVE (P0)

✓ Search
- SHOULD-HAVE (P1)

› Uploads

ID	Capability	Acceptance Signal
FR1	Search	FTS

Non-Functional Requirements

SLOs, not vibes

06

The quality attributes that define production-readiness — each with a measurable target and the mechanism that enforces it.

Attribute	Target (SLO)	Enforcement Mechanism
Perf	Fast	PG

QUALITY ATTRIBUTE PRIORITY



Tech Stack Decisions

Every choice has a bill

07

Node + Postgres.

Layer	Chosen	Why This	Trade-off Accepted
DB	PG	FTS	Scaling

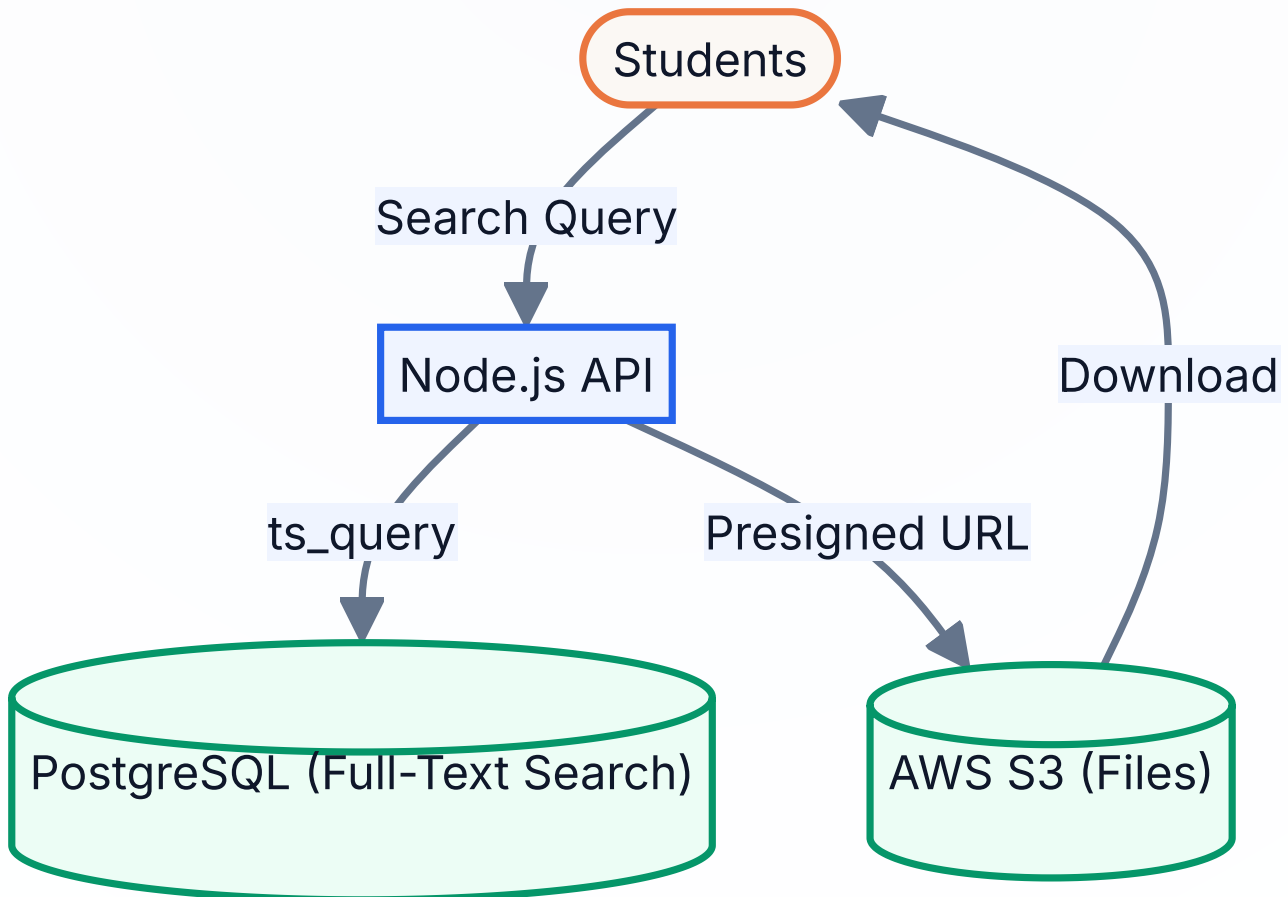
Architecture Overview

Diagram 1 — System Architecture

08

● Sankalan — System Architecture

C4 / CONTAINER



Client / Edge Service Data store Async / Queue External API

○ ERASER DIAGRAM SPEC

Students [icon: users]
API [icon: nodejs, color: blue]
Postgres [icon: postgresql, color: green]
S3 [icon: aws, color: green]

Students > API: search
API > Postgres: full-text search
API > S3: get presigned URL
S3 > Students: download

■ COMPONENT BREAKDOWN

Postgres FTS Leverages ts_vector for instant document retrieval without Elasticsearch.

S3 Presigned URLs Secure, direct downloads offloading bandwidth from the API.

▣ VISUAL LAYOUT DESCRIPTION

Top-to-bottom search and retrieve flow.

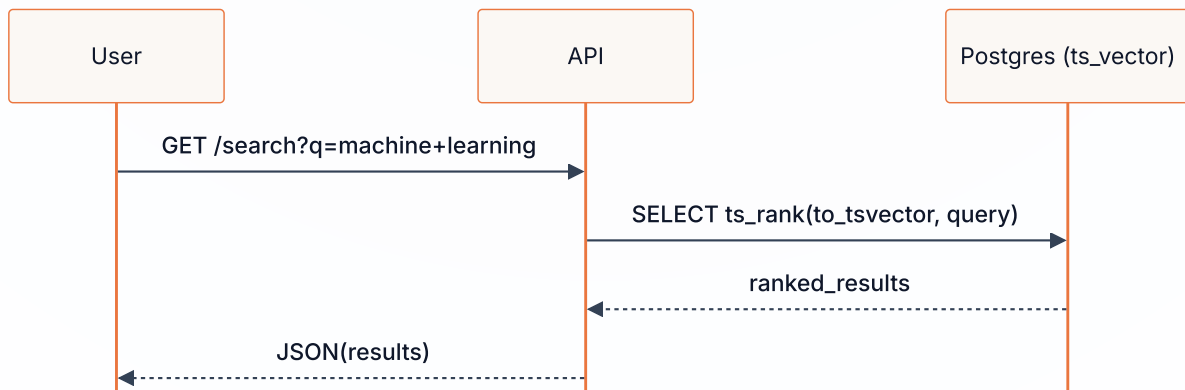
Request Lifecycle

Diagram 2 — End-to-End Request Flow

09

● Search Flow

SEQUENCE



○ ERASER DIAGRAM SPEC

User > API: GET /search
API > DB: ts_rank query
DB > API: ranked results
API > User: JSON

■ COMPONENT BREAKDOWN

ts_rank Orders results by term frequency and relevance.

▣ VISUAL LAYOUT DESCRIPTION

Standard API sequence.

Database Design

Diagram 3 — Entity Relationship Model

10

Sankalan — Data Model

ERD

RESOURCE		
uuid	id	PK
string	title	
string	url	
tsvector	text_search_index	

ERASER DIAGRAM SPEC

RESOURCE { id pk; title; url; text_search_index }

COMPONENT BREAKDOWN

tsvector Precomputed index column for fast queries.

VISUAL LAYOUT DESCRIPTION

Simple.

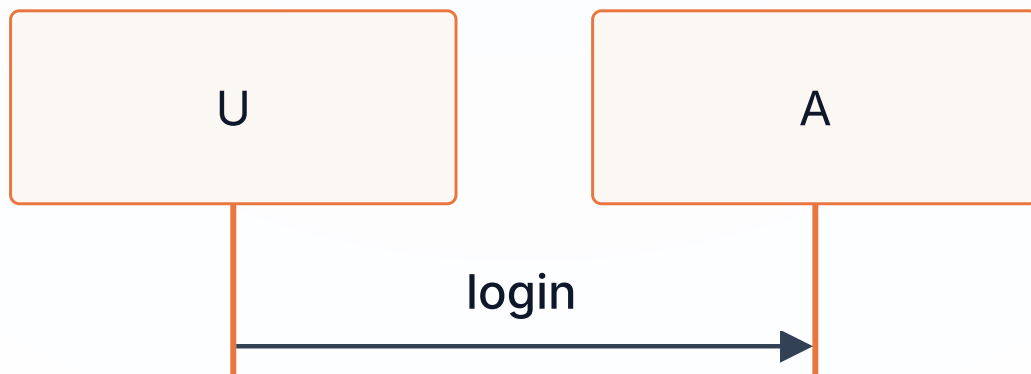
Authentication Design

Diagram 4 — Identity & Authorization Flow

11

● Auth

SEQUENCE



ERASER DIAGRAM SPEC

U > A

COMPONENT BREAKDOWN

OAuth

Google Sign-In.

VISUAL LAYOUT DESCRIPTION

Standard

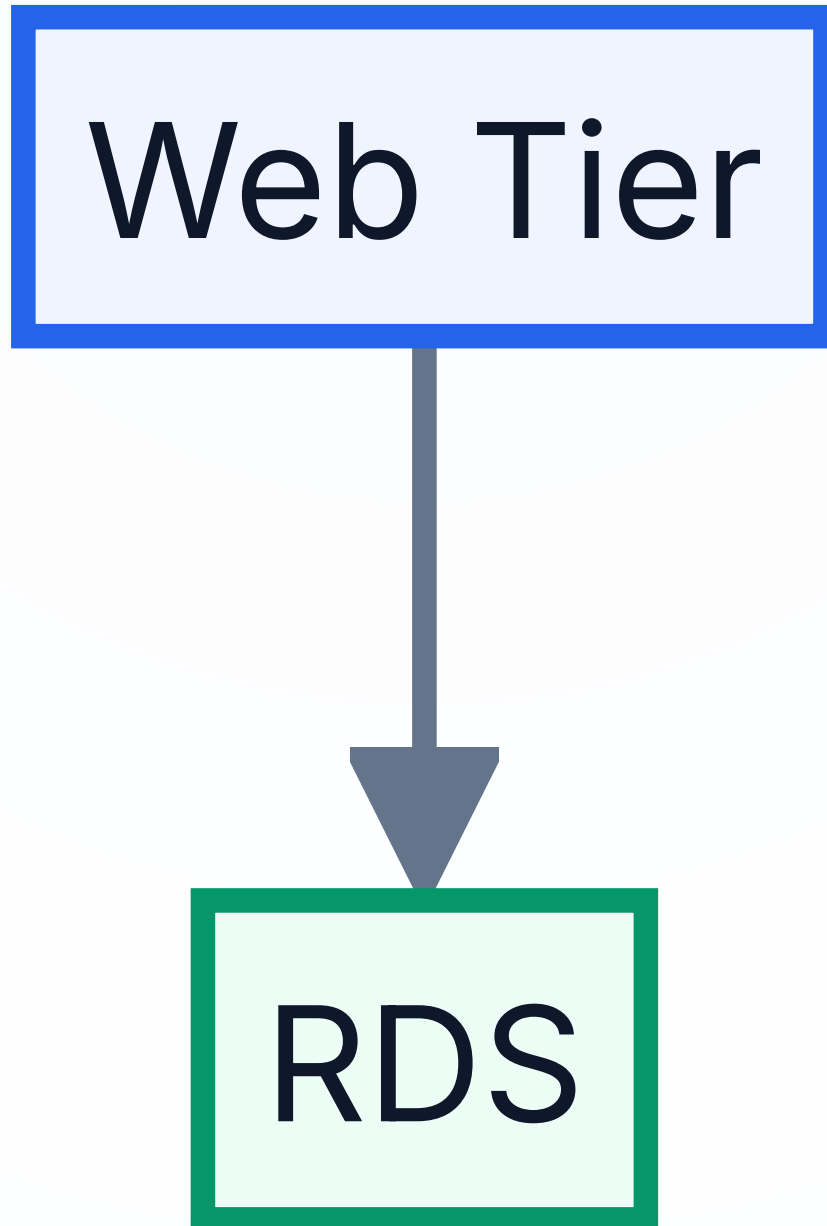
Deployment Architecture

Diagram 5 — Cloud Deployment Topology

12

Deploy

DEPLOYMENT



Client / Edge Service Data store Async / Queue External API

ERASER DIAGRAM SPEC

W > DB

COMPONENT BREAKDOWN

RDS

Managed Postgres.

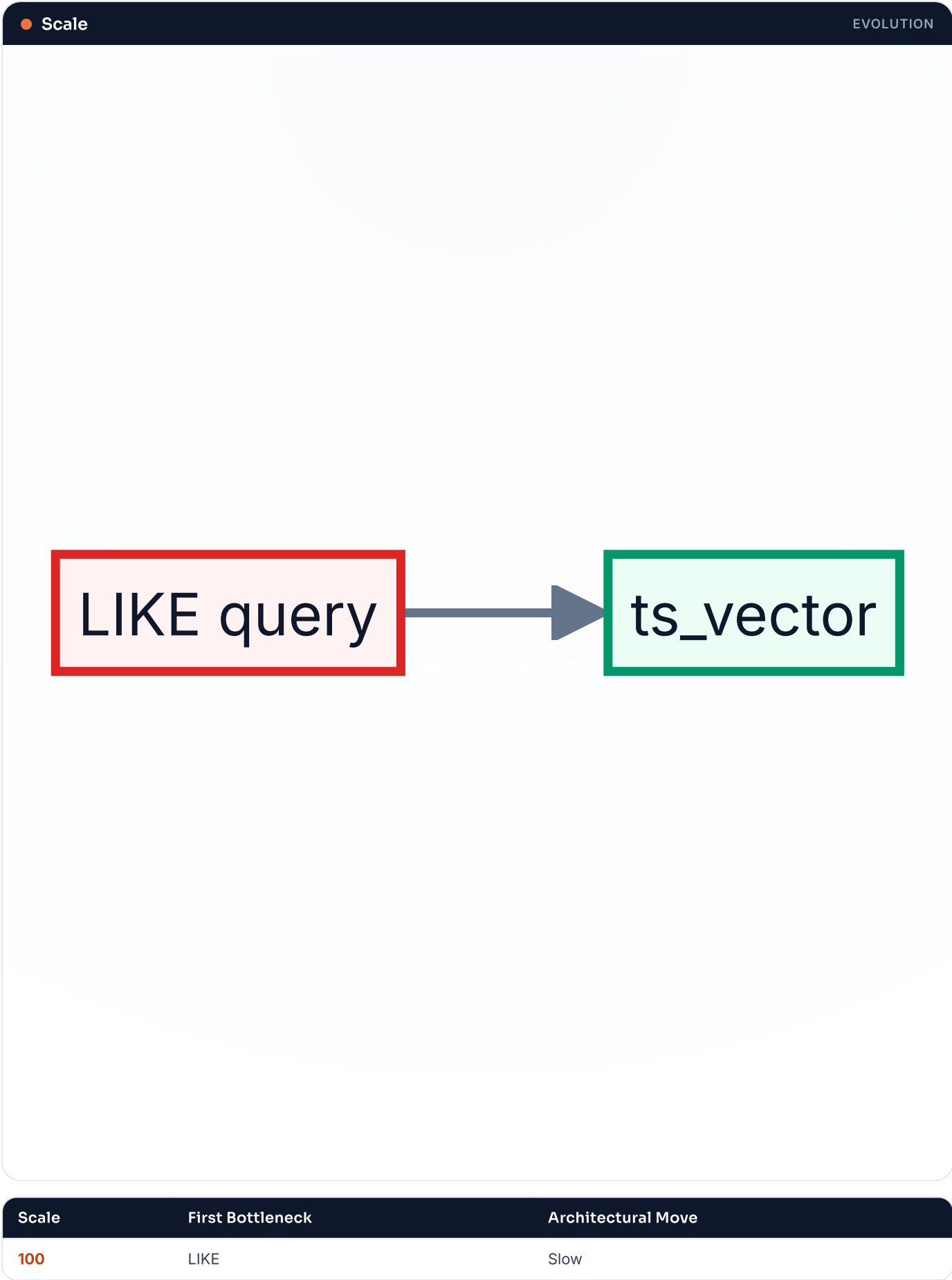
VISUAL LAYOUT DESCRIPTION

Standard

Scalability Architecture

Diagram 6 — Scaling Evolution 100 → 1M

13



Presigned URLs.

Threat (STRIDE)	Vector	Mitigation
Leak	Presigned URL	Expiry

 **S3 security** — Use short-lived pre-signed URLs.

Failure Recovery

Diagram 7 — Failure & Recovery Topology

15

● Failure

RESILIENCE

DB Slowdown

Cache

Redis

● Healthy ● Failed / Degraded ● Recovery path

ERASER DIAGRAM SPEC

DB > Redis

COMPONENT BREAKDOWN

Query Cache Caches frequent searches.

VISUAL LAYOUT DESCRIPTION

Resilience.

Monitoring & Observability

Golden signals + alert matrix

16

Instrumented around the four golden signals. Every alert is actionable and maps to a runbook.



Search Lat
ms

ALERTING MATRIX

Signal	Threshold	Action / Runbook
Slow Query	Index	

SANKALAN · SECTION 17

Cost Optimization

Unit economics under load

Interview Yield ★★☆☆☆

17

No ElasticSearch.

Low

COST

LEVERS PULLED

 Avoid ES

Engineering Challenges

What broke, why, and what we learned

18

PRODUCTION INCIDENT — POST-MORTEM



The Incident

Unindexed query



Root Cause

Missing index



The Fix

Add GIN index



Lasting Lesson

Always use GIN



Blast radius — Slowdowns.

Architecture Tradeoffs

Chosen sacrifices, stated plainly

19

Every architecture is a set of deliberate sacrifices. Here are the ones we made on purpose — and what we gave up.

Decision	We Chose	We Sacrificed
PG FTS	ElasticSearch	Simplicity

WHERE WE OPTIMIZED

Simplicity  Max

SANKALAN · SECTION 20

Future Roadmap

Where the architecture goes next

Interview Yield ★★☆☆☆

20

AI summaries.

V2

AI

Summaries.

Interview Deep Dive

The questions that decide the loop

21

Q Why PG FTS?

Simplicity.

FTS

SANKALAN · SECTION 22

Interview Yield ★★★★★

Revision Sheet

One-page recall before the round

22

Postgres FTS.

ARCH

API → PG FTS

**60-second pitch** — Sankalan makes academic files instantly searchable using Postgres Full-Text Search.

